

IMPERIAL

IMPERIAL COLLEGE LONDON

DEPARTMENT OF AERONAUTICAL ENGINEERING

FINAL YEAR INDIVIDUAL REPORT

Vision Transformer Masked Autoencoders for Restricted-Domain Flow Reconstruction

Author:

Ben Eze

CID:

02039117

Supervisor:

Asst. Prof. Andrea Nóvoa

Co-Supervisor:

Prof. Luca Magri

1st June, 2026

Abstract

Vision Transformers have demonstrated outstanding performance for image generation, yet their adoption within fluid dynamics remains limited. The *Latent Attention on Masked Patches* (LAMP) model is thus introduced as an interpretable single-layer Vision Transformer, modified for training with regression. LAMP operates by patching the masked input snapshot and projecting it into a latent space through Proper Orthogonal Decomposition, where it can efficiently make predictions, before projecting back into the physical domain. Both the Vision Transformer Masked Autoencoders (ViT-MAE) and LAMP architectures are investigated and compared for masked flow reconstruction – the process of recovering the full flow state from partial measurements – on three 2D unsteady wakes: the laminar wake past a bluff body, the chaotic wake past two cylinders and the turbulent wake behind a porous flat plate. On the laminar wake, LAMP is able to reconstruct and filter the flow-field from 90%-masked and noisy input across signal-to-noise ratios from 10 to 30 dB. The learned attention maps serve as an interpretable multi-fidelity guide for optimal sensor-placement. LAMP’s limitations on the chaotic two-cylinder wake motivate the need for the ViT-MAE, which is able to reconstruct with a mean-squared-error of 0.058, almost an order of magnitude lower than LAMP. Operating on patches results in visible artefacts, however, the addition of a U-Net block is shown to remove these. The most notable flaw with the current approach is that it generates intermittent but significant loss spikes arising from the ill-posed nature of the inverse problem, and introducing temporal history is proposed as a remedy. On the turbulent wake, data augmentation is shown to be essential, enabling reconstruction beyond just the mean flow, despite a limited dataset size.

Acknowledgements

I would like to thank my supervisor Andrea N. for the guidance she provided throughout the project, and for co-authoring and encouraging me to publish a paper on the initial LAMP results. Thank you also to my second supervisor Luca M., who also advised on and co-authored the LAMP paper, and with whom I will be continuing my research journey in the form of a PhD.

And to Ioana, thank you for keeping me company during all those late night study sessions, for your encouragement and help understanding POD. And finally to my flatmates for waiting for me to finish this report when we should have been at the pub two hours ago.

List of Symbols

Symbols

$\mathbf{A}$	Log-attention matrix
$\mathbf{a}$	Activation function
$\mathbf{b}$	Bias vector
C	Number of channels
D	Decoder
D_{cyl}	Cylinder diameter
$\mathbf{D}$	Linear decoder matrix
d	Latent dimension
d_h	Dimension per attention head ($d = N_h d_h$)
d_{MLP}	MLP hidden dimension
d_P	Patch dimension ($d_P = P^2 C$)
d_x	Dimension of input vector $\mathbf{x}$
E	Encoder
$\mathbf{E}$	Linear encoder matrix
$\mathbf{E}_{pos}, \mathbf{e}_{pos}$	Positional encoding
$f(\cdot)$	Generic function
f^{unmasked}	Fraction of data unmasked
H	Height of input field
$\mathbf{I}$	Identity matrix
$\mathbf{K}$	Key matrix
$\mathcal{L}$	Loss
$\mathcal{L}^{\text{POD}}$	Autoencoder loss
$\mathcal{L}^{\text{lin}}$	Linearisation loss
$\mathcal{L}^{\text{pred}}$	Masked prediction loss
N	Number of patches
$\mathcal{N}$	Normal distribution
$\mathcal{N}^{\text{mask}}$	Mask indices
N_h	Number of attention heads
N_l	Number of attention layers
P	Patch size

Continued on next page

P_{noise}	Noise power
$\mathbf{Q}$	Query matrix
Re	Reynolds number
R^2	Coefficient of determination
T	Number of time steps/snapshots
Δt	Time step
t	Time
u	Streamwise velocity
$\mathcal{U}$	Uniform distribution
$\mathbf{U}^\top$	SVD left singular vectors (i.e. mode shapes)
U_∞	Free-stream velocity
v	Wall-normal velocity
$\mathbf{V}$	Value matrix
$\mathbf{V}^\top$	SVD right singular vectors
W	Width of input field
$\mathbf{W}$	Weight matrix
$\mathbf{W}^A$	Attention-weight tensor
$\mathbf{W}^{ED}$	Encoder/decoder projection matrix
$\mathbf{W}^{emb}$	Embedding-weight matrix
$\mathbf{W}^K$	Key-weight matrix
$\mathbf{W}^O$	Projection matrix (in Multi-Head Self-Attention)
$\mathbf{W}^Q$	Query-weight matrix
$\mathbf{W}^{un}$	Unembedding matrix
$\mathbf{W}^V$	Value-weight matrix (ViT) or tensor (LAMP)
w	Spanwise velocity
x	Horizontal position
$\mathbf{X}, \mathbf{x}$	Input variable
$\mathbf{X}_{in}$	Input tensor
y	Vertical position
$\mathbf{Z}, \mathbf{z}$	Latent variable

Greek Letters

β_ω	Vorticity loss coefficient
ϵ	Noise
μ	Mean
Σ	SVD matrix of singular values
σ	Standard deviation
Ω	Domain
$\omega_z, \omega_z(\cdot)$	z -component of vorticity/2D vorticity, vorticity operator

Super/sub-scripts

$(\cdot)^{\text{crop}}$	Cropped
$(\cdot)^{\text{dec}}$	Decoder
$(\cdot)^{\text{enc}}$	Encoder
$(\cdot)^{\text{mask}}$	Of a mask
$(\cdot)^{\text{masked}}$	Masked quantity
$(\cdot)^{\text{noisy}}$	Noisy
$(\cdot)^{\text{train}}$	Training
$(\cdot)^{\text{test}}$	Test
$(\cdot)^{\text{val}}$	Validation
$(\cdot)_i, (\cdot)_j$	Index (generic subscript)
$(\cdot)_l$	Layer index (in model)
$(\cdot)_m$	Patch index (row)
$(\cdot)_n$	Patch index (column)

Other

$\tilde{(\cdot)}$	Quantity decoded from latent representation
$\hat{(\cdot)}$	Data pre-standardisation
$(\cdot)^*$	Predicted quantity
$(\cdot)'$	Velocity perturbation
$\bar{(\cdot)}$	Mean quantity
$\mathbf{X}$	Matrix quantity
$\mathbf{x}$	Vector quantity
$\mathcal{X}$	Set or space

Acronyms

AE	Autoencoder
CAE	Convolutional Autoencoder
CFD	Computational Fluid Dynamics
CFL	Courant-Friedrichs-Lewy
CNN	Convolutional Neural Network
DNS	Direct Numerical Simulation
ESRGAN	Enhanced Super-Resolution Generative Adversarial Network
GAN	Generative Adversarial Network
LAMP	Latent Attention on Masked Patches
LES	Large Eddy Simulation
MAE	Masked Autoencoder
MH SA	Multi-Head Self-Attention
MLP	Multi-Layer Perceptron
MSE	Mean Squared Error
PC	Principal Component
PCA	Principal Component Analysis
PDF	Probability Density Function
PIV	Particle Image Velocimetry
POD	Proper Orthogonal Decomposition
PSD	Power Spectral Density
ROM	Reduced Order Model
SNR	Signal-to-Noise Ratio
ViT	Vision Transformer

Contents

Abstract	i
Acknowledgements	iii
1 Introduction	3
1.1 Motivation	3
1.2 Existing Methods	4
1.3 Aims and Objectives	5
2 Background Theory	7
2.1 Autoencoders	7
2.1.1 Proper Orthogonal Decomposition (POD)	7
2.1.2 Multi-Layer Perceptrons (MLPs)	8
2.2 Gappy POD	9
2.3 Transformers	10
2.3.1 Vision Transformers (ViTs)	11
2.3.2 Vision Transformer Masked Autoencoders (ViT-MAEs)	11
3 Methodology	13
3.1 LAMP	13
3.1.1 Embedding Layer (Patch-wise POD)	14
3.1.2 Masked Reconstruction	14
3.2 Datasets	15
3.2.1 Dataset 1 - Periodic Triangular-Body Wake	15
3.2.2 Dataset 2 - Two-Cylinder Chaotic Wake	15
3.2.3 Dataset 3 - Porous Plate LES Wake	16
3.2.4 Data Augmentation	16
3.3 Training	18
3.3.1 Experimentation Framework	18
3.3.2 Hyperparameter Tuning	18
3.3.3 Training Schedule	19

3.3.4	Artefact Removal	19
3.4	Evaluation	20
3.4.1	Reconstruction Mean Squared Error	20
3.4.2	Noise Removal	21
3.4.3	Distribution Plots	21
4	Results and Discussion	22
4.1	Dataset 1 (LAMP vs Gappy POD)	22
4.1.1	POD Reconstruction Loss	22
4.1.2	Masked Reconstruction	22
4.1.3	Noise Removal	23
4.1.4	Attention Maps	24
4.2	Dataset 2 (LAMP, Gappy POD and ViT-MAE)	25
4.2.1	POD Reconstruction Loss	26
4.2.2	Masked Reconstruction (LAMP vs Gappy POD)	26
4.2.3	LAMP Attention Map	27
4.2.4	ViT-MAE Hyperparameter Tuning	28
4.2.5	ViT-MAE Time Series and Anomaly Analysis	29
4.2.6	ViT-MAE Noise Filtering	30
4.2.7	Artefact Removal	32
4.2.8	Distributions Comparison	32
4.2.9	Speed Comparison	34
4.3	Dataset 3 (ViT)	35
4.3.1	POD Reconstruction Loss	35
4.3.2	Mean-Flow Plateau	36
4.3.3	Domain Size	37
4.3.4	Hyperparameter Tuning	38
4.3.5	Data Augmentation	38
4.3.6	Masked Reconstruction	39
5	Conclusion	40
5.1	Summary of Thesis Achievements	40
5.2	Applications	40

5.3 Future Work	41
Bibliography	41
A Experimentation Framework	45
B Dataset 3 ViT-MAE Hyperparameters	48
C Distribution Correlation (R^2) Values	49

List of Tables

3.1	Default model hyperparameters for the ViT-MAE on Dataset 2, obtained from scaling down the architecture in [25]. The model contains a total of 32,551,778 trainable parameters.	19
3.2	Default training hyperparameters for ViT-MAEs on Dataset 2. Values in [25] were adjusted for the smaller training set.	19
4.1	$\mathcal{L}^{\text{pred}}$ for a range of patch sizes, identifying $P = 24$ as the optimal value. The maximum and minimum values across three runs are shown, giving an indication of the spread of results.	28
4.2	Results for a parameter sweep over the width, N_h , and depth, N_l , of the attention blocks, width of the MLP, d_{MLP} , power of training noise, SNR, degree of dropout and number of samples. The grey refers to the default settings and bold highlights the lowest loss per table.	29
4.3	Training and inference time for the three models used in the results section. Training time is reported for the dataset with 8,000 samples for comparison between the three methods (and scales linearly with the number of snapshots) and inference time is calculated for a single sample without batching. An AMD Ryzen Threadripper PRO 5995WX (64-core, 128-thread) CPU is used for the Gappy POD and LAMP results and ViT-MAE uses and a single NVIDIA GeForce RTX 4090 GPU (24 GB VRAM).	34
4.4	$\mathcal{L}^{\text{pred}}$ is calculated for ViT-MAEs with a range of patch sizes, demonstrating that $P = 18$ is optimal. Results quoted are for <i>random</i> patch arrangements.	38
B.1	Architecture hyperparameters used for the ViT training on Dataset 3. The total parameter count is 44,393,864	48
B.2	Training hyperparameters used for the ViT training on Dataset 3.	48
C.1	R^2 scores for flow-statistic reconstruction of LAMP. The R^2 value for the PSD is given for a linear and log scale.	49

C.2 R^2 scores for flow-statistic reconstructions of Gappy POD, LAMP and ViT-MAE.

The R^2 value for the PSD is given for a linear and log scale. 49

List of Figures

- 2.1 POD applied to Dataset 1. (a) shows an example snapshot of the flow, below it the mean flow (i.e. mode zero) and the first two modes or 'principal components' (PCs). (b) shows the trajectory of the flow in POD space, which periodically traces out circles when projected onto planes formed by neighbouring pairs of PCs. From the plot of variance explained in (c), one can observe that i) with just a couple of modes, we can reconstruct the majority of the variance of the flow data ii) the modes come in pairs iii) when noise of increasing signal-to-noise ratio (SNR) is added, it proportionally affects the higher modes first, drowning out the high-resolution content but retaining large structures. 8
- 2.2 Top: (a) shows a diagram of the ViT-MAE architecture, split into encoder and decoder. Bottom: a more detailed view of (b) patch embedding, (c) the encoder/decoder bridge, (d) multi-head self-attention and (e) patch unembedding. . 12
- 3.1 Illustration of the LAMP model: A linear autoencoder embeds the input into a lower-dimensional latent space, where predictions can be made efficiently and decodes the output. Predictions made in the latent space and are pair-wise (each patch predicts every other patch), and are weighted with the attention matrix, calculated from the input data. 13
- 3.2 Summary of the three datasets. (a) Dataset 1: laminar wake over a triangular bluff body with $Re = 100$, (b) Dataset 2: chaotic two-cylinder wake with $Re = 200$ and (c) Dataset 3: LES simulation of a turbulent wake behind a porous flat plate with $Re = 6,000$. For each, the two velocity components u, v and vorticity w_z are shown. The dashed rectangles represent the training and validation domains. For Datasets 1 and 2, $\Omega^{\text{val}} = \Omega^{\text{train}}$, and for Dataset 3, $\Omega^{\text{val}} \subset \Omega^{\text{train}}$ (coloured red and blue respectively), as discussed in Section 4.3.3. Datasets 1 and 3 each use a single time series (with an interval removed between the training and validation snapshots), whereas Dataset 2 contains eight independent time series, each seeded with random noise, of which seven are used for training, one for validation and one for testing. 17

- 4.1 Patch-wise POD reconstruction error for varying patch size P and embedding dimension d . Autoencoder loss $\mathcal{L}^{\text{POD}}$ is plotted against (a) latent dimension, d and (b) compression factor, d_P/d . The grey line shows the reconstruction accuracy of a global POD basis. 22
- 4.2 Reconstruction error, $\mathcal{L}^{\text{pred}}$, is presented for a range of patch sizes and embedding dimensions, calculated from the median of 25 random patch arrangements, and $f^{\text{unmasked}} = 0.1$ is used. On the right is an example reconstruction with $P = 16$, $d = 8$, demonstrating that the reconstruction is indistinguishable from the ground truth by eye with very few modes. 23
- 4.3 Key velocity distributions along the lines $x/W = 0.625$ (a, b, d) and the centreline, $y = 0$ (c), for flow-field reconstructed by the LAMP model which achieved the lowest $\mathcal{L}^{\text{pred}}$ (with $P = 48, d = 24$). The normalised (a) RMS u , (b) RMS v , (c) centreline recovery and (d) Reynolds shear stress distributions are all plotted and $R^2 = 1$ to within numerical precision for all four distributions, showing a perfect reconstruction. 23
- 4.4 The probability density function for each velocity perturbation component (a) u' and (b) v' is calculated at the point $x/W = 0.625$ on the centreline. Again, the distribution for the reconstructed field exactly matches the ground truth. The reason for the asymmetry in the ground truth v' distribution is simply because the number of snapshots tested is not a multiple of the oscillation period. . . . 24
- 4.5 The power spectral density for (a) u' and (b) v' is calculated, again at $x/W = 0.625, y = 0$. (a) shows slight disagreement in the PSDs at the highest frequencies, and in (b) this discrepancy is even smaller. Accurate capturing of the PSD shows that LAMP can accurately capture the dynamics of the flows it reconstructs. . . 24
- 4.6 Reconstruction error $\mathcal{L}^{\text{pred}}$ is plotted for varying noise levels, patch sizes P and latent dimensions d , all with 10% patches unmasked. The input is noisy masked data with $SNR =$ (b) 30 dB, (c) 20 dB, (d) 10 dB. The loss is computed against the noise-free target $\mathbf{X}^{\text{test}}$ and the model is trained on noise-free data. Horizontal dashed lines show the noise variance. 25

4.7	Attention maps for varying P and d , calculated taking a mean over columns of the log-attention matrix $\mathbf{A}$. The latent flow state in brighter regions is able to, on average, predict the rest of the patches with lower loss. This offers a data-driven strategy for optimal multi-fidelity sensor placement.	25
4.8	Autoencoder loss is plotted against the (a) embedding dimension and (b) compression factor. The loss descends less steeply with increasing d than with Dataset 1.	26
4.9	Masked reconstruction of the chaotic wake from 25% unmasked patches, all located within the region $0.25 < x/W < 0.5$. The reconstruction of one snapshot with the best LAMP model ($P = 48, d = 250$) is shown below. The Gappy POD reconstruction shown is not from the lowest-loss model, to illustrate the difference between the tentative LAMP and confident Gappy POD predictions. The same qualitative behaviour is observed throughout the range of latent dimensions tested.	27
4.10	Attention maps for $P = 16$. Values range from 1.6 to only 3.0 from $d = 4$ to 250 compared to 6 to 15 from $d = 6$ to 12 in Fig 4.7. This represents a much higher uncertainty (the log-attention plotted is the average <i>negative</i> log error of pair-wise linear predictions).	28
4.11	An example flow-field reconstructed by a ViT-MAE with $P = 24$, all other parameters given in Table 3.1, showing a significant improvement on LAMP and Gappy POD (Fig. 4.9)	28

- 4.12 Anomaly analysis of the ViT-MAE reconstruction error across all test samples. (a) shows the test MSE (blue) and the Mahalanobis distance of the patch embedding from the centroid of the training embedding (red). The negligible $R^2 = 8 \times 10^{-4}$ value suggests that the loss peaks are not associated with out-of-distributed inputs at the patch embedding state. The five highest local maxima of the $\mathcal{L}^{\text{pred}}$ curve are found and labelled (1 – 5). (b) shows projections of the patch embedding onto the first two principal components (i.e. POD modes) of the training distribution (grey) with validation snapshots coloured by $\log(\mathcal{L}^{\text{pred}})$. Stars indicate the projection of the five loss peaks and the diamonds show their nearest-neighbour training samples in PCA space. Peaks 1 and 5 are used for the snapshot comparison in (c), which shows the following vorticity fields, from left to right: difference between the validation snapshot and its nearest training neighbour, the nearest neighbour, the validation snapshot, ViT-MAE reconstruction of the validation snapshot and reconstruction error. The flow-field in the input window matches closely between the validation snapshot and nearest neighbour, but differs significantly downstream. 31
- 4.13 Probability distributions of per-snapshot MSE for two ViT-MAEs tested on Dataset 2 with added noise. (a) shows the results of the baseline model (trained on clean data) and (b) uses a model with the same architecture, but trained on noisy 30 dB training data. The model trained on noisy data performs slightly worse on the low noise data, but better on 20 and 30 dB noise. The distributions resemble a log-normal distribution (note that the x-axis is logarithmic). 32
- 4.14 The vorticity plots reconstructed by the (a) reference ViT-MAE and (b) ViT-MAE with U-Net are shown. The reconstructed vortices are smoother and the error plot does not contain the grid artefacts. 33
- 4.15 Distributions calculated in the same way as in Fig. 4.3 – also along the lines $x/W = 0.625$ and $y = 0$ – from the velocity fields reconstructed by the lowest- $\mathcal{L}^{\text{pred}}$ model variants of each class: LAMP (red) with $P = 48, D = 300$, Gappy POD (blue) with $D = 10$ and ViT-MAE (green) with the *reference architecture* (Table 3.1) and $P = 24$ 34

- 4.16 Probability density functions for (a) u' and (b) v' , calculated as in Fig. 4.4 at the point $x/W = 0.65, y = 0$. (a) shows a very positive skew, indicating the presence of high bursts of increased u . For (b), due to the symmetry of the setup about the line $y = 0$, the v' distribution should tend to a perfectly symmetrical distribution with the addition of more validation samples. 35
- 4.17 Power spectral density of u' and v' for the reconstructed fields, calculated as in Fig. 4.5 at the point $x/W = 0.65, y = 0$ 35
- 4.18 Patch-wise POD reconstruction loss plotted against (a) number of POD modes (b) compression factor. In contrast to the equivalent plots for datasets 1 and 2, (a) shows a massive increase in the number of modes required to reconstruct with the same accuracy. In (b), the curves from successive patch sizes cross over, showing that, to ensure low enough $\mathcal{L}^{\text{POD}}$, it becomes more efficient to use smaller patches. The flow in distant regions have such low correlation that it is more efficient to split up the domain and represent them with independent bases 36
- 4.19 Loss plateau experienced by models of varying architectures. (a) shows the training (grey) and validation (blue) loss curves for a range of models – none of them improve upon the loss if one were to just predict the mean flow (shown by the red dashed line). Hyperparameters d, N_l, N_h, P , number of epochs, learning rate and weight decay were all varied. (b) shows an example snapshot reconstructed by one of these models, which can be compared with the actual mean flow calculated on the training data – they appear very similar. 37
- 4.20 The effect of the training domain Ω_{train} on the loss. (a) shows how increasing the number of samples monotonically decreases the loss, (b) and (c) show the decrease in loss that can be obtained by increasing the width and height of the training domain respectively, and (d) illustrates the random crop technique. The validation domain Ω^{val} is fixed and then training samples Ω^{crop} are uniformly sampled from Ω^{train} 38
- 4.21 ViT-MAE reconstruction on Dataset 3 with input windows of equal widths and spacing of length (a) 1.125 m (b) 2.25 m (c) 4.5 m. The input is shown on the top row, with the reconstruction below and columnwise MSE below that. 39

- A.1 Example dashboard showing the loss curves of all the models in a parameter sweep. The reconstruction from the lowest loss model is also shown. The script `run_dashboard.py` continuously runs and updates the figure `dashboard.png` and the PAUSED indicates that the script has reached the time limit. 47

1 - Introduction

1.1 Motivation

Within various engineering and scientific disciplines, knowledge of the current state of a flow-field is essential for predicting or controlling its future evolution. Consider, for example, the problem of modelling the turbulent air flow-field over a road vehicle to feed into an active flow control system for drag reduction, using only surface pressure readings [1]; or combining sparse measurements of velocity, pressure, temperature and humidity across continents to construct a picture of the global atmospheric state for weather prediction [2]; or extrapolating a full three-dimensional flow-field from two-dimensional (2D) Particle Image Velocimetry (PIV) [3] slices recorded in a wind tunnel test. All of these scenarios share a common challenge: the need to form a useful representation of a flow-field from incomplete observations. This problem is fundamentally ill-posed [4] – the mapping from the partial observations to the full flow state is one-to-infinitely-many – but physically plausible solutions form only a small subset of those consistent with the input data.

This challenge is then compounded by the typically high spatial dimensionality of fluid flows. Flow structures exist across a wide range of length scales – from the Kolmogorov microscales of turbulent dissipation right up to large-scale geophysical phenomena – meaning that *fully* resolving even a single snapshot in time requires an unfeasibly large number of grid points. Consequently, full-state measurements are rarely feasible in practice. In wind tunnel testing, for instance, one may have access only to a few dozen pressure taps, the fluorescent oil-film technique [5], or video data of tufts on a wing surface [6], from which the broader flow-field must be inferred. A more powerful experimental tool is Particle Image Velocimetry (PIV) [3], a non-intrusive technique in which tracer particles are introduced into the flow, a planar laser sheet is used to illuminate them, and image processing of pairs of successive frames yields the local velocity field. While PIV can provide high-resolution data, it struggles in high-speed flows and, in most setups, is limited to a 2D cross-section; this is a significant restriction given the inherently three-dimensional nature of turbulence.

The curse of dimensionality extends equally to resolving the temporal behaviour. Turbulent flows are chaotic – predictions become unreliable beyond a certain time horizon – which necessitates high temporal resolution and real-time data assimilation to track the flow’s evolution accurately. A full CFD simulation resolves the governing equations on a fine grid and is far too computationally expensive for real-time use. The traditional alternative is to employ simplified models derived from first principles – such as panel methods or vortex lattice methods – which trade accuracy for speed by making significant assumptions about the nature of the flow. These assumptions break down, though, in challenging conditions, such as a wing operating at high angles of attack.

The modern, data-driven solution to both the inverse problem and issue of dimensionality is to use reduced order models (ROMs) [7]: These learn a compact, yet faithful latent representation of the full flow state. If the latent representation can be identified from incomplete measure-

ments, decoding it recovers the flow state at unobserved locations. Additionally, the compression makes forecasting tractable: the latent space evolution can be simulated efficiently, before projecting the final state back into the physical domain [8] or, in the case of a control problem, fed directly into a controller [1].

In summary, there is a clear need for data-driven methods capable of forming an efficient latent representation of a flow-field from limited measurements, from which the full flow state can be accurately reconstructed. The following section reviews the existing methods developed to this end.

1.2 Existing Methods

Deep learning has seen remarkable successes over the past decade. Of particular relevance to this work is the rapid progress in models for processing image and video data (typically the task of classifying or generating it). From the development of Convolutional Neural Networks (CNNs) [9] in the 1980s, originally applied to the classification of handwritten digits, to AlexNet [10] kick-starting the modern deep learning era, the field has advanced at a remarkable pace. Generative Adversarial Networks (GANs) [11] produced some of the first photorealistic generated images, and more recently diffusion models [12, 13] and Vision Transformers (ViTs) [14] have pushed the state of the art further still, capitalising on the vast quantities of training data available. The relevance of these advances to fluid dynamics lies in a simple equivalence: a flow snapshot can be treated as an image – instead of three colour channels one considers the velocity components u , v , w , or other relevant quantities such as pressure, temperature, density.

Proper Orthogonal Decomposition (POD) [15] is arguably the simplest ROM, decomposing flow snapshots into a linear combination of orthogonal spatial modes ranked by energy content. Its simplicity, reliability, and ease of implementation make it a natural baseline upon which to build. Gappy POD [16], for instance, applies linear regression on top of a POD basis to recover full snapshots from sparse measurements. Many *nonlinear* architectures have also built upon this foundation: for example [17] combines POD with CNNs to reconstruct bulk flow structures from just shear stress measurements at the wall.

CNNs are particularly well suited to image-like data, owing to their high inductive bias: principles of locality and spatial invariance are both baked into CNN architectures. This often results in far fewer trainable parameters, making them easier to train. These properties have made CNNs a natural choice for Convolutional Autoencoders (CAEs); [18] demonstrates that a CAE can represent a turbulent three-dimensional flow-field to the same accuracy as POD with just 10% as many modes. CNNs have also been widely applied to super-resolution, in which low-resolution data is upsampled to recover fine-scale detail; this is both used to enhance experimental measurements [19, 20] and to reconstruct the subgrid-scale closure terms in Large Eddy Simulation (LES) from coarse-grained data [21], effectively bridging the gap between LES and Direct Numerical Simulation (DNS). A limitation of convolutional layers, however, is their inherently local receptive field; despite their efficiency at extracting *local* features, they struggle to capture long-range spatial dependencies.

GANs have similarly found application in flow reconstruction, notably for recovering three-dimensional velocity fields from 2D measurements [22]. A variant, the Enhanced Super-Resolution Generative Adversarial Network (ESRGAN), has been applied to reconstruct flow outside the field of view of PIV windows [23]. A known limitation of GAN-based approaches, however, is the generation of artefacts when applied to heavily masked data, and one this project directly aims to address.

A recent contender to GANs for such tasks is the Vision Transformer (ViT). ViTs minimally adapt the Transformer architecture [24] to handle images by first splitting them into a grid of non-overlapping patches, which replace tokenised text data. The key innovation that distinguishes Transformer-based models from other architectures is the attention mechanism, which captures long-range dependencies that convolutions, by design, cannot. Since their introduction [14], ViTs have shown a remarkable capacity to learn from large quantities of training data, outperforming CNNs when trained at scale; the drawback, however, is that they *require* larger datasets compared to CNNs. Learning from limited data is a core challenge for scientific applications, where high-quality data is more expensive to generate than in computer vision. A central aim is thus to identify methods that allow ViTs to perform competitively on limited data.

The original paper introducing ViTs [14] proposed using them for classification, with masked patch prediction introduced as a secondary objective. This idea was then developed further in [25], which showed that ViTs can reconstruct images in which up to 90% of the pixel information has been removed. Importantly, in the paper this masked reconstruction task was just a *proxy* for learning efficient latent representations of the target classes, so a perfect reconstruction is not the end goal; the results still demonstrate a striking ability to reconstruct high-level concepts from very limited observations. ViTs have since gone on to replace U-Nets as the backbone of Diffusion Transformers [26], placing them at the core of modern text-to-image generation [27]. The success of ViTs across image classification, masked reconstruction and generation suggests that the architecture is capable of learning the fundamental structure of high-dimensional data, a property that this work aims to explore within the context of fluid flow.

1.3 Aims and Objectives

The aim of this project is to reproduce complete 2D (u, v) snapshots from static, masked input using a Vision Transformer Masked Autoencoder (ViT-MAE) and the newly proposed LAMP model, where LAMP was developed for this project to build intuition about concepts such as attention, and to benchmark the performance of full ViT-MAEs. A ‘good’ ViT-MAE should...

- **Achieve low reconstruction loss.** First and foremost, it should predict the flow in the masked regions with low mean-squared error, without adding significant artefacts, specific to the architecture used. In a practical application such as forecasting or flow control, an MAE is only one component in a larger framework, and different applications may impose more specific requirements (such as smoothness of the latent space or observability), however in this work, low per-pixel reconstruction loss is found to be a reliable indicator of good performance across other evaluation metrics.
- **Train well with limited data.** High quality experimental and CFD data is far less abundant than the datasets used to train large vision models. Deep learning models are

inherently massively over-parametrised, so careful model sizing and data augmentation will be tested to tackle this problem, allowing the models enough capacity to learn the highly complex and nonlinear flows, whilst delaying over-fitting.

- **Be robust to noise.** Real-world data is noisy and models should be able to distinguish between noise and signal, both on training and test data. Different approaches, including POD and adding noise to training data will be explored to ensure that the models can reconstruct 30 dB noise, which is a proxy for experimental noise, as discussed in Section 3.4.2.
- **Be fast at inference.** In real-time applications, especially those involving control, speed is of significant importance. The transformer architecture is, by design, efficient at operating in parallel, utilising the batch dimension well [24]. However in deployment, waiting to group snapshots over multiple time steps before evaluating would result in an unacceptable time lag. Different architectures are thus tested against each other on speed.

To demonstrate their performance, the models are tested across three unsteady wakes of increasing complexity. A laminar bluff body wake with a Reynolds number $Re = 100$ is used as the *a priori* case, to show that even the simplified LAMP model can demonstrate near-perfect reconstruction accuracy on 90%-masked data with a periodic flow *and* that the method is robust to noisy test data. LAMP and ViT-MAE are then tasked with reconstructing a chaotic wake past two cylinders at $Re = 200$, a task which shows the limitations of LAMP and necessity for the more complex and deeper attention blocks present in ViT-MAEs. The final test case involves applying ViTs to the wake behind a porous plate at a much higher $Re = 6,000$. Due to both the complexity of the flow and limited number of samples, the models are unable to resolve details within the wake, however domain sizing and data augmentation are investigated which improve the reconstructions beyond simply predicting the mean flow.

The rest of this report is organised in the following sections. Chapter 2 explains the relevant Machine Learning theory to give context to the models developed and tested. Chapter 3 introduces the LAMP architecture developed for this project, plus details about the data, training and evaluation. Chapter 4 presents and discusses the results of the tests across the three datasets and Chapter 5 concludes the report, summarising the findings.

2 - Background Theory

2.1 Autoencoders

Autoencoders play an important role in both flow analysis – reducing the dimensionality of the flow state vector into something more usable – and for reconstructing images – specifically with the use of *masked* autoencoders. In the general case, autoencoding is an unsupervised machine learning technique in which a model is tasked with reconstructing an input vector via an intermediate *latent* variable, but with certain constraints such that it cannot just learn the identity matrix $\mathbf{I}$; as a by-product, they may develop certain desirable properties (e.g. compression or noise removal). Written formally, we want functions $E : \mathbb{R}^{d_x} \rightarrow \mathbb{R}^d$ (the encoder), $D : \mathbb{R}^d \rightarrow \mathbb{R}^{d_x}$ (the decoder), such that

$$\mathbf{z} = E(\mathbf{x}), \quad \tilde{\mathbf{x}} = D(\mathbf{z}), \quad \tilde{\mathbf{x}} \approx \mathbf{x}, \quad (2.1)$$

where we must define what it means for $\tilde{\mathbf{x}} \approx \mathbf{x}$. To implement this, we define a loss function for the autoencoder $\mathcal{L}^{\text{AE}}(\tilde{\mathbf{x}}, \mathbf{x}) : \mathbb{R}^{d_x}, \mathbb{R}^{d_x} \rightarrow \mathbb{R}$, which measures the dissimilarity between the reconstruction and input. The autoencoder is then trained on data to learn the functions E and D , with the aim of reconstructing similar, unseen data with low loss. Generally, the loss function is the mean squared error, so formulating this as an optimisation problem, we seek to minimise

$$\mathcal{L}^{\text{AE}} = \frac{1}{d_x} \|\tilde{\mathbf{x}} - \mathbf{x}\|_2^2, \quad \text{subject to} \quad \tilde{\mathbf{x}} = D(E(\mathbf{x})) \quad (2.2)$$

It is important that the training data distribution closely matches that of test data – since E is not invertible (for $d < d_x$), autoencoders can only reconstruct inputs *close* to the latent manifold.

In the case of high-dimensional flow data, we want our latent variable $\mathbf{z}$ to be a compressed representation of $\mathbf{x}$, ($d \ll d_x$), whilst still containing the key physical information. If this is the case, we may then forecast the future state of $\mathbf{z}(t)$ over the window $t \in [t, t+T]$. For sufficiently large d_x , the added computational cost of the encoding and decoding is outweighed by the cost saved by advancing the state of a lower-dimensional system $[\mathbf{z}(t), \mathbf{z}(t+\Delta t), \dots, \mathbf{z}(t+T)]$.

Alternatively, we may have corrupted (e.g. masked or noisy) observations, of the state vector. For masked reconstruction, the hope is that there is enough information for the encoder to form a faithful latent representation to decode, and if not, the reconstruction should at least be consistent with the input and the training distribution. In the case of noise, because autoencoders are typically bottlenecked, the model is incentivised by the loss function to retain the high-variance signal, discarding noise, which is random and uncorrelated.

2.1.1 Proper Orthogonal Decomposition (POD)

So far, the description of autoencoders has neglected the actual definition of functions E and D . The simplest method is to use linear functions for both of these, namely $E(\mathbf{x}) = \mathbf{E}\mathbf{x}$, $D(\mathbf{z}) = \mathbf{D}\mathbf{z}$, for some matrices $\mathbf{E} \in \mathbb{R}^{d \times d_x}$, $\mathbf{D} \in \mathbb{R}^{d_x \times d}$. Gradient descent *could* be used, but if the aim is to reduce the mean squared error, a closed-form solution exists: POD (also known

as Principal Component Analysis or Singular Value Decomposition) finds the best (i.e. most variance explained) rank- d approximation of input $\mathbf{X}$. Namely,

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top = \mathbf{U}\mathbf{Z}, \quad (2.3)$$

where matrix $\mathbf{X} \in \mathbb{R}^{d_x \times T}$ contains T vectors $\mathbf{x}$ (in our case, T snapshots of the flow state), $\mathbf{U} \in \mathbb{R}^{d_x \times d}$ contains the d orthonormal mode shapes or *basis functions*, $\mathbf{\Sigma} \in \mathbb{R}^{d \times d}$ is a diagonal matrix where each element squared, Σ_i^2 , is proportional to the variance explained by mode i , and $\mathbf{V} \in \mathbb{R}^{d \times T}$ contains the time series of each mode in the latent space. The workflow is therefore as follows: compute $\mathbf{U}$ on the training data, giving us the decoder matrix $\mathbf{D} = \mathbf{U}$; the encoder matrix is just the inverse of this, $\mathbf{E} = \mathbf{U}^{-1}$, and because $\mathbf{U}$ is orthonormal, $\mathbf{U}^{-1} = \mathbf{U}^\top$. Figure 2.1 shows a demonstration of POD on Dataset 1. For periodic flows, POD captures the behaviour of the flow very accurately with just a few modes: 99% of the variance is recovered with just the first 6 modes – note that this is not typically the case in turbulent flows (e.g. [15] showed that a backward-facing step flow required 50 modes to capture just 75% of the turbulent energy). Figure 2.1 (b) shows the regular trajectory of the flow in the latent space and (c) shows how noise affects the higher modes first.

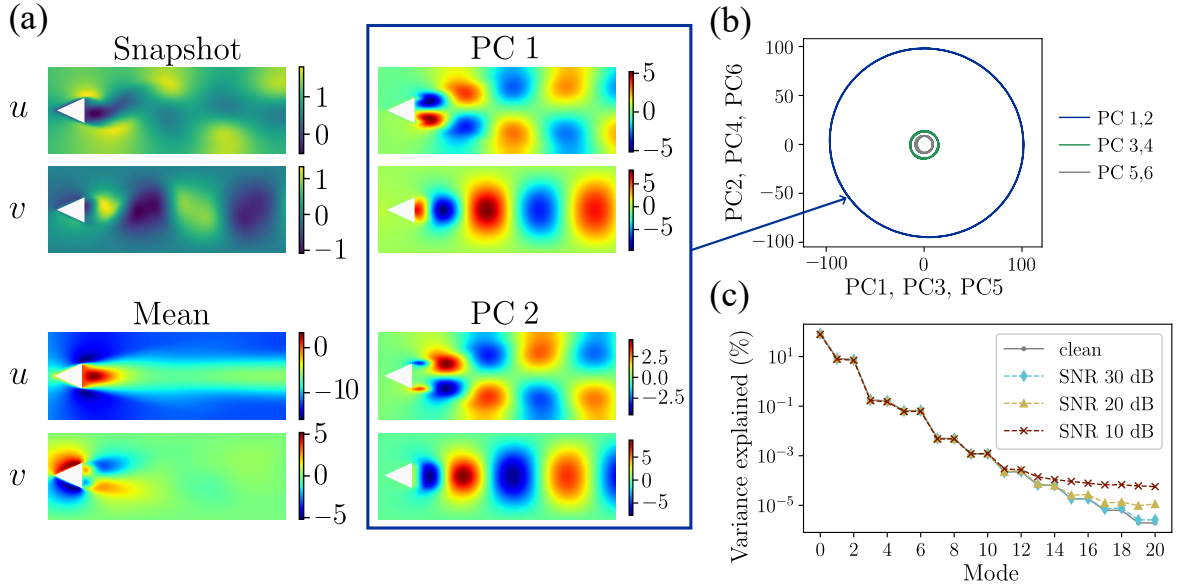


Figure 2.1: POD applied to Dataset 1. (a) shows an example snapshot of the flow, below it the mean flow (i.e. mode zero) and the first two modes or 'principal components' (PCs). (b) shows the trajectory of the flow in POD space, which periodically traces out circles when projected onto planes formed by neighbouring pairs of PCs. From the plot of variance explained in (c), one can observe that i) with just a couple of modes, we can reconstruct the majority of the variance of the flow data ii) the modes come in pairs iii) when noise of increasing signal-to-noise ratio (SNR) is added, it proportionally affects the higher modes first, drowning out the high-resolution content but retaining large structures.

2.1.2 Multi-Layer Perceptrons (MLPs)

In machine learning the go-to model (or component) to represent a generic function $f : \mathbb{R}^{d_x} \rightarrow \mathbb{R}^d$ is the MLP [28], due to its relative simplicity and property as a universal function approximator [29]. In short, it is formed of N_l layers through which the input passes sequentially, and each layer involves multiplying with weight matrix $\mathbf{W}$, addition of a bias vector $\mathbf{b}$ and applying a

nonlinear piecewise activation function $\mathbf{a}$. Concretely, to get from the value at each layer $\mathbf{x}_l$ to the next, $\mathbf{x}_{l+1}$,

$$\begin{aligned} MLP(\mathbf{x}) : \quad & \mathbf{x}_{l+1} = \mathbf{a}(\mathbf{W}_l \mathbf{x}_l + \mathbf{b}_l), \\ \text{for} \quad & \mathbf{W}_l \in \mathbb{R}^{d_l \times d_{l+1}}, \quad \mathbf{b}_l \in \mathbb{R}^{d_{l+1}}, \quad \mathbf{a}_i : \mathbb{R} \rightarrow \mathbb{R}, \quad l \in [0, N_l] \end{aligned} \quad (2.4)$$

and MLPs can be used for both the encoder and decoder functions. Though simple to implement and despite their theoretical power for approximating any function, in practice MLPs can be hard to train for deep models [30]. Every element in every layer is connected to every other element in the next layer, and this parameter density has undesirable consequences: they are computationally expensive and sensitive to initialisation, there is no inductive bias (i.e. everything must be learned from scratch), and they are hard to interpret. Therefore autoencoders rarely use one single MLP for E or D , however shallow MLPs are still present as components *within* many autoencoder architectures (e.g. CAEs and ViT-MAEs) operating on intermediate representations.

2.2 Gappy POD

Perhaps the simplest physically-informed method to reconstruct state vectors from masked measurements is Gappy POD [16]. The full gappy POD framework, further developed in [31] includes sensor placement optimisation for robustness to noisy data, however this is not discussed here, since the input data (namely rectangular PIV windows) is already prescribed. The method works as follows:

Assume we are studying a system with a state vector $\mathbf{x} \in \mathbb{R}^{d_x}$. We have access to an array, $\mathbf{X}^{\text{train}} \in \mathbb{R}^{d_x \times T^{\text{train}}}$, of T^{train} full-state snapshots of $\mathbf{x}$ in training but at test time, we are only able to measure $\mathbf{X}^{\text{masked}}$, a subset of the full state $\mathbf{X}^{\text{test}}$ (both $\in \mathbb{R}^{d_x \times T^{\text{test}}}$). We can write $\mathbf{X}^{\text{masked}} = \mathbf{I}^{\text{mask}} \mathbf{X}^{\text{test}}$, where $\mathbf{I}^{\text{mask}}$ is the diagonal ‘masked identity matrix’, with diagonal entries

$$I_i^{\text{mask}} = \begin{cases} 1 & \text{if } \mathbf{X}_i^{\text{test}} \text{ is known} \\ 0 & \text{if } \mathbf{X}_i^{\text{test}} \text{ is missing} \end{cases} \quad (2.5)$$

We can then form the masked mode shape matrix $\mathbf{U}^{\text{masked}} = \mathbf{I}^{\text{mask}} \mathbf{U}$. With this we can then simply use linear regression to solve for the latent space representation $\mathbf{Z}^{\text{test}}$, before projecting back into the physical space:

$$\mathbf{Z}^{\text{test}} = \arg \min_{\mathbf{Z}} \|\mathbf{U}^{\text{masked}} \mathbf{Z} - \mathbf{X}^{\text{masked}}\|_2^2 \quad (2.6)$$

$$\tilde{\mathbf{X}}^{\text{test}} = \mathbf{U} \mathbf{Z}^{\text{test}} \quad (2.7)$$

In short, Gappy POD is a least-squares approximation in a data-driven basis, where the basis functions, $\mathbf{U}$, are determined from POD on the training snapshots. It works very well on simple flows with a high variance explained by a few POD modes and is robust to noise due to the noise-removing property of POD. One flaw of the method, however, is that it provides no uncertainty estimate, always returning a single confident prediction. When the flow state lies far outside the training distribution, the reconstruction can be wildly inaccurate. This will be tackled in

Section 3.1.

2.3 Transformers

The backbone of most recent developments in machine learning since their introduction in 2017 [24], the transformer has proven itself to learn exceptionally well, across a wide range of tasks, especially when provided with large amounts of data.

The architecture can be divided into three main components: i) the embedding layer converts the input (tokens in the context of language models) into a sequence of latent vectors, ii) the N_l transformer layers carry out the prediction calculations in the latent space and iii) the unembedding layer converts back into a useful output (Fig. 2.2 illustrates a modified version of this within ViT-MAEs). The key innovation of transformers is the multi-headed self attention mechanism within the attention block and it is this part of the architecture which remains almost identical between different transformer architectures, regardless of the modality. The next section describes how Vision Transformers (ViTs) apply the same architecture to image data.

To understand transformer layers, we must study the ‘scaled dot-product attention’ equation, written in terms of query, key and value matrices ($\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d_h}$), each of which are linear functions of $\mathbf{Z}$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_h}}\right) \mathbf{V} \quad (2.8)$$

$$\mathbf{Q} = \mathbf{Z}\mathbf{W}^{\mathbf{Q}}, \quad \mathbf{K} = \mathbf{Z}\mathbf{W}^{\mathbf{K}}, \quad \mathbf{V} = \mathbf{Z}\mathbf{W}^{\mathbf{V}}, \quad \mathbf{W}^{\mathbf{Q}}, \mathbf{W}^{\mathbf{K}}, \mathbf{W}^{\mathbf{V}} \in \mathbb{R}^{d \times d_h} \quad (2.9)$$

The log-attention matrix $\mathbf{A} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}$ contains the affinities between each ordered pair of tokens¹. The softmax operation then rescales the matrix so that all elements are positive and each row sums to unity, so it can appropriately scale each value in $\mathbf{V}$ to pass to the next layer.

With a single self-attention ‘head’, we have one communication pathway, but by concatenating the output of multiple parallel attention heads and projecting their outputs with projection matrix $\mathbf{W}^{\mathbf{O}} \in \mathbb{R}^{d \times d}$, transformers can attend to different aspects of the input simultaneously:

$$\begin{aligned} \text{MultiHead}(\mathbf{Z}) &= \text{Concat}(\text{head}_1, \dots, \text{head}_{N_h}) \mathbf{W}^{\mathbf{O}} \\ \text{where } \text{head}_i &= \text{Attention}(\mathbf{Z}\mathbf{W}_i^{\mathbf{Q}}, \mathbf{Z}\mathbf{W}_i^{\mathbf{K}}, \mathbf{Z}\mathbf{W}_i^{\mathbf{V}}) \end{aligned} \quad (2.10)$$

Once every patch has ‘communicated’ with every other patch through the multi-headed self-attention mechanism, the output is added onto the residual stream. This is then passed through an MLP with one hidden layer of dimension d_{MLP} with the GeLU (Gaussian Error Linear Unit):

$$\text{Block}(\mathbf{Z}) : \begin{cases} \mathbf{Z} \leftarrow \mathbf{Z} + \text{MultiHead}(\mathbf{Z}) \\ \mathbf{Z} \leftarrow \mathbf{Z} + \text{MLP}(\mathbf{Z}) \end{cases} \quad (2.11)$$

The residual connection ($\mathbf{Z} \leftarrow \mathbf{Z} + f(\mathbf{Z})$ instead of $\mathbf{Z} \leftarrow f(\mathbf{Z})$) ensures an uninterrupted pathway

¹For an intuitive understanding, we can interpret $\mathbf{Q}$ as sequence of vectors stating “I am looking for information $\mathbf{q}_m$ ” and $\mathbf{K}$ representing “I contain information $\mathbf{k}_n$ ”, then each element of the log-attention matrix contains scores of how aligned these question-answer pairs are, $\mathbf{A}_{mn} = \mathbf{q}_m \cdot \mathbf{k}_n$.

along which gradients can flow during back-propagation, which greatly improves the ease of optimisation with gradient descent, particularly with deep models [32].

2.3.1 Vision Transformers (ViTs)

Introduced in [14], ViTs have demonstrated themselves to be remarkably powerful, however the key design principle behind their success was to modify as little of the original Transformer architecture as possible. Not only is the self-attention mechanism able to learn complex relationships between tokens, but also Transformer algorithmic and hardware improvements, originally intended for Large Language Models (LLMs) have often carried over to ViTs [33].

The key idea of ViTs is to split our domain into non-overlapping square patches and operate on those (see Fig. 2.2 (a)). The normalised input 2D data (of height H , width W and C channels), $\mathbf{X}_{\text{in}} \in \mathbb{R}^{H \times W \times C}$, is split into N square patches of dimension $d_P := P \times P \times C$, which are flattened to obtain the patched data matrix $\mathbf{X} \in \mathbb{R}^{N \times d_x}$. These flattened patches are then projected into a latent space of dimension d with embedding matrix $\mathbf{W}^{\text{emb}} \in \mathbb{R}^{d_x \times d}$, which remains the dimension of the residual stream throughout the transformer block. Since the attention mechanism is order-invariant, the spatial position of patches must be encoded with a positional encoding matrix $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{N \times d}$. Within this project, $\mathbf{E}_{\text{pos}}$ is treated as a learned parameter; a sinusoidal positional encoding can be specified, but was found to yield no performance improvement [14]. Once the patch embedding is applied, the result is $\mathbf{Z} \in \mathbb{R}^{N \times d}$, which can be treated just like the token embedding matrix in LLM Transformers. The next section demonstrates how ViTs can be used as masked autoencoders: in short, ViT-MAEs use the embedding layer and transformer blocks from ViTs as an encoder, reintroduce a mask token for each masked patch and mirror the encoder structure to reconstruct the full image.

2.3.2 Vision Transformer Masked Autoencoders (ViT-MAEs)

In [25], ViT-MAEs are explored as a *proxy* for learning a meaningful representation of images, as a way to pre-train classifiers. The idea is that, whilst natural language is already a heavily compressed format – very few words can contain a lot of information – individual pixels in image data, on the other hand, contain far less information, with large portions of the pixels being redundant². To force the model to learn an *efficient* internal representation of the input data, the data is first heavily masked (up to 95%) and trained to produce ‘semantically plausible’ reconstructions. In [25], the decoder part of the architecture is then discarded and replaced with a classifier to operate on the learned embeddings, but in this work, the decoder is retained for reconstruction.

ViT-MAEs use the same patch embedding block and transformer layers as in a standard ViT, except that just the unmasked patches are input; this is the encoder (operating on d^{enc} -dimensional vectors). There is then an encoder-decoder bridge, which connects the encoder and decoder residual streams ($\mathbf{Z}^{\text{enc}}$ and $\mathbf{Z}^{\text{dec}}$ respectively), passing on encoded patches (where visible) and a mask token where masked.

²A picture may be “worth a thousand words”, but if it is a high definition image of your pet dog, containing 1280×720 pixels with 3 colour channels, it is no longer as informationally efficient (at least in the context of classification) as a simple *alt text* description **a brown dog with floppy ears**.

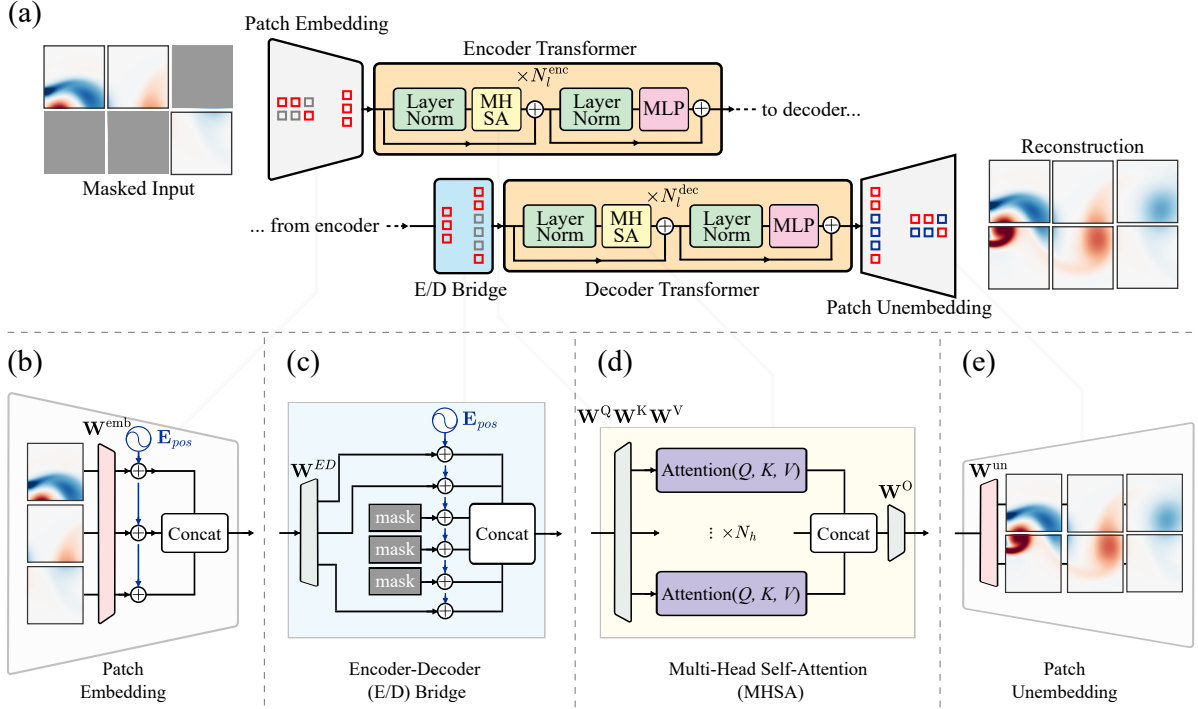


Figure 2.2: Top: (a) shows a diagram of the ViT-MAE architecture, split into encoder and decoder. Bottom: a more detailed view of (b) patch embedding, (c) the encoder/decoder bridge, (d) multi-head self-attention and (e) patch unembedding.

$$\mathbf{Z}^{\text{dec}} = \text{Bridge}(\mathbf{Z}^{\text{enc}}) \quad \text{where} \quad (2.12)$$

$$\text{Bridge}(\mathbf{z}_n^{\text{enc}}) = \begin{cases} [\text{mask}] + (\mathbf{e}_{\text{pos}}^{\text{dec}})_n & \text{if } n \in \mathcal{N}^{\text{mask}} \\ \mathbf{z}_n^{\text{enc}} \mathbf{W}^{\text{ED}} + (\mathbf{e}_{\text{pos}}^{\text{dec}})_n & \text{otherwise} \end{cases} \quad (2.13)$$

The mask token, $[\text{mask}] \in \mathbb{R}^{d^{\text{dec}}}$ is a trainable, shared vector, projection matrix $\mathbf{W}^{\text{ED}} \in \mathbb{R}^{d^{\text{enc}} \times d^{\text{dec}}}$ projects from d^{enc} to the decoder dimension d^{dec} , and to reintroduce the position, an additional positional embedding $\mathbf{E}_{\text{pos}}^{\text{dec}} \in \mathbb{R}^{N \times d^{\text{dec}}}$ is added.

With all N patches present, a decoder-transformer is added. At each subsequent layer, information aggregates into the residual stream of the masked patch positions to form a representation of the flow at that patch.

Finally, a single linear layer projects the output of the decoder transformer block back into patches in the physical space with unembedding matrix $\mathbf{W}^{\text{un}} \in \mathbb{R}^{d^{\text{dec}} \times d_P}$.

3 - Methodology

3.1 LAMP

To gain intuition about ViTs and provide a new, interpretable, simplified benchmark, more relevant to patch-based methods than Gappy POD, the *Latent Attention on Masked Patches* (LAMP) model for flow reconstruction was developed [34]. It takes inspiration from ViT-MAEs, whilst being trained with POD and two sets of linear regression – this makes the results fast and reproducible – and there are only two parameters to modify: patch size and embedding dimension. Due to the modularity of the architecture, it can easily be modified to use more expressive components where necessary, such as nonlinear autoencoders.

As illustrated in Fig. 3.1, the overall structure resembles that of the ViT but has the following key differences:

- **Local POD embedding.** Each patch has its own independent, local POD embedding – it requires fewer modes for accurate reconstruction, but a fixed domain must be used.
- **Single attention layer with one SA head.** LAMP uses a single attention head ($N_l = N_h = 1$) so that the value- and attention weight matrices can be given a concrete meaning: the value weight matrices are defined as pairwise linear predictors, and the attention weights reflect the confidence of each prediction. The two sets of matrices can then be trained sequentially with linear regression.
- **Pair-specific weight matrices.** Since LAMP uses a single linear attention layer, it cannot accumulate positional context the way a deep ViT can. Instead, positional information is encoded by learning a separate set of weight matrices for each ordered pair (m, n) .

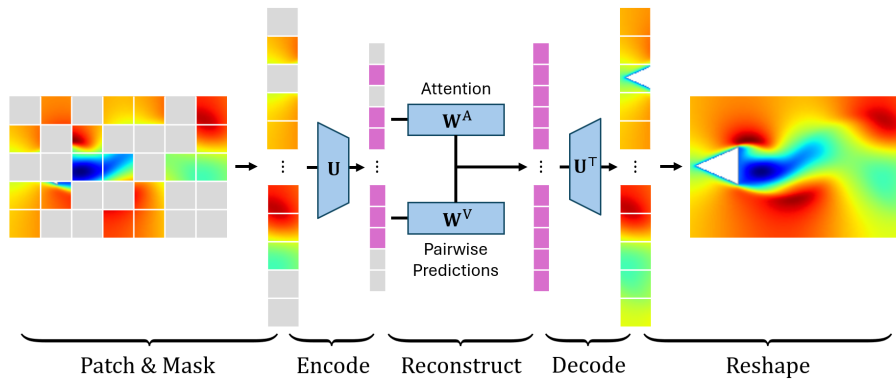


Figure 3.1: Illustration of the LAMP model: A linear autoencoder embeds the input into a lower-dimensional latent space, where predictions can be made efficiently and decodes the output. Predictions made in the latent space are pair-wise (each patch predicts every other patch), and are weighted with the attention matrix, calculated from the input data.

⁰All code for this project can be found in <https://github.com/Ben-Eze/CylinderFlow.git>

3.1.1 Embedding Layer (Patch-wise POD)

The first step of patching the normalised input is the same as described in Section 2.3.1 so the embedding step takes as input $\mathbf{X} \in \mathbb{R}^{N \times d_P}$. Patch-wise POD is then applied to compress $\mathbf{X}$ into $\mathbf{Z} \in \mathbb{R}^{N \times d}$, with latent dimension $d \ll d_P$. Each patch n has associated with it mode shapes $\mathbf{U}_n$, found by minimising the POD autoencoder loss,

$$\mathcal{L}^{\text{POD}} = \frac{1}{d_x} \sum_{n=1}^N \|\tilde{\mathbf{x}}_n - \mathbf{x}_n\|_2^2, \quad \text{subject to} \quad \tilde{\mathbf{x}}_n = \mathbf{U}_n \mathbf{U}_n^\top \mathbf{x}_n. \quad (3.1)$$

This minimisation is applied on training data $\mathbf{X}_n^{\text{train}} \approx \mathbf{U}_n \boldsymbol{\Sigma}_n (\mathbf{V}_n^{\text{train}})^\top$, where the approximation arises from only retaining the first d leading singular vectors. Finally we construct our encoding and decoding operators $\mathbf{E}$ and $\mathbf{D}$ as block diagonal matrices, with diagonal entries $\mathbf{U}_n^\top$, $\mathbf{U}_n$ respectively, such that

$$\mathbf{Z} = \mathbf{E}\mathbf{X}, \quad \text{and} \quad \tilde{\mathbf{X}} = \mathbf{D}\mathbf{Z}. \quad (3.2)$$

3.1.2 Masked Reconstruction

Once the patched latent representation, $\mathbf{Z}$, is formed, we would like a way to reconstruct a full latent snapshot $\mathbf{Z}^*$ from masked input $\mathbf{Z}^{\text{masked}}$, where $Z_n^{\text{masked}} = Z_n$ for all *unmasked* indices $n \notin \mathcal{N}^{\text{mask}}$ and $Z_n = \mathbf{0}$ elsewhere (akin to Gappy POD). Each component in $\mathbf{Z}^*$ is

$$Z_{me}^* = \sum_{n=1}^N \text{softmax}(\mathbf{a}_m)_n V_{mne}, \quad \text{for} \quad \begin{cases} m = 1, \dots, N, \\ e = 1, \dots, d \end{cases} \quad (3.3)$$

where $\mathbf{a}_m$ is the m -th row of the log-attention matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$, and $\mathbf{V} \in \mathbb{R}^{N \times N \times d}$ is the value tensor. These are defined as

$$A_{mn} = \sum_{e=1}^d W_{mne}^A Z_{ne}^{\text{masked}}, \quad (3.4)$$

$$V_{mne} = \sum_{f=1}^d W_{mnef}^V Z_{nf}^{\text{masked}}, \quad (3.5)$$

where $\mathbf{W}^V \in \mathbb{R}^{N \times N \times d \times d}$ and $\mathbf{W}^A \in \mathbb{R}^{N \times N \times d}$ are the value and attention weight-tensors¹, which are trained on *unmasked* data with a two-step approach:

1. The value weight tensor $\mathbf{W}^V$ can be viewed as an $N \times N$ grid of matrices: each block $\mathbf{W}_{mn}^V \in \mathbb{R}^{d \times d}$ defines a linear map that predicts $\mathbf{z}_m^*$ from $\mathbf{z}_n \quad \forall m, n \in [1, N], n \neq m$. This acts just like $\mathbf{W}^V$ in Eqn 2.8, except these blocks are obtained pair-wise and with least-squares regression, minimising the *linearisation* loss

$$\mathcal{L}_{mn}^{\text{lin}} = \|(\mathbf{Z}^{\text{train}})_m - \mathbf{W}_{mn}^V (\mathbf{Z}^{\text{train}})_n\|_2^2. \quad (3.6)$$

2. The attention tensor $\mathbf{W}^A$ is learned in a second stage and can be decomposed analogously,

¹This is different from the definition of log-attention in the Transformer architecture (Eqn. 2.8). Because LAMP makes its predictions in a single layer, it cannot use any information accumulated at patch index m to inform A_{mn} . Instead, only the *position* of patch m and the position and *content* of patch n can be used to predict how to the weight prediction (m, n) from the value matrix.

as a $N \times N$ grid of vectors $w_{mn}^A \in \mathbb{R}^d$. The error $\mathcal{L}_{mn}^{\text{lin}}$ provides a prediction uncertainty measure, so that predictions with lower $\mathcal{L}_{mn}^{\text{lin}}$ are assigned higher attention weights, and with $\mathbf{W}^A$ we can linearly predict it on unseen data. In practice, we regress the *log-error*,

$$\mathbf{W}_{mn}^A = \arg \min_{\mathbf{W}} \left\| \mathbf{W}(\mathbf{Z}^{\text{train}})_n - (-\log(\mathcal{L}_{mn}^{\text{lin}})) \right\|_2^2, \quad (3.7)$$

since A_{mn} can take both positive *and* negative values, and softmax converts these into confidence score which sum to unity. At inference, we set the log-error, $A_{mn} \leftarrow -\infty \quad \forall \quad n \in \mathcal{N}^{\text{mask}}$ to ensure that softmax assigns them zero attention.

When evaluating the model on unseen test data $\mathbf{X}^{\text{test}}$, the overall prediction error is defined as

$$\mathcal{L}^{\text{pred}} = \left\| \tilde{\mathbf{X}}^* - \mathbf{X}^{\text{test}} \right\|_2^2 = \left\| \mathbf{D}\mathbf{Z}^* - \mathbf{X}^{\text{test}} \right\|_2^2. \quad (3.8)$$

3.2 Datasets

Three 2D wake datasets, with progressively higher Reynolds numbers and complexity, are used to test the two models. All three datasets contain the streamwise u and wall-normal v velocity fields on a rectangular domain at even time intervals. The datasets are standardised component-wise to allow for a valid comparison between datasets.

$$X_{tjic} = \frac{\hat{X}_{tjic} - \mu_c}{\sigma_c}, \quad \text{for } \begin{cases} t = 1, \dots, T, & j = 1, \dots, H, \\ i = 1, \dots, W, & c = 1, \dots, C, \end{cases} \quad (3.9)$$

where μ_c, σ_c are the mean and standard deviation per component, and $(\hat{\cdot})$ denotes unnormalised data. The transient behaviour is discarded, so only the statistically steady-state data is used. The remaining data is split into training and test data, with an intermediate portion removed to prevent temporal data leakage. The domains Ω are also cropped to the training, domain Ω^{train} , to remove the bluff body and excess free-stream. Figure 3.2 summarises all three datasets and their pre-processing.

3.2.1 Dataset 1 - Periodic Triangular-Body Wake

The setup behind Dataset 1 is the periodic laminar wake behind a triangular bluff body with a Reynolds number $Re = 100$. The dataset contains 200 snapshots at time intervals $\Delta t = 0.125$, of which the first 100 are used for training, and the last 60 for validation. One shedding period is approximately 32 snapshots. Since no hyperparameters are tuned on LAMP and Gappy POD (used on Dataset 1), the validation set serves as the test set. For a more detailed breakdown of the CFD setup, see [35].

3.2.2 Dataset 2 - Two-Cylinder Chaotic Wake

To trigger chaotic behaviour, Dataset 2 involves the flow behind *two* cylinders of diameter $D_{cyl} = 1$ unit, with their centres spaced 1.5 units apart. The whole simulation was performed with the 2D compressible Navier Stokes equations on a domain $\Omega \in [-3, 17] \times [0, 12]$ with velocity inlet ($u = 1, v = 0$), pressure outlet and periodic side boundary conditions, however just

the portion of the wake $\Omega^{\text{val}} = \Omega^{\text{train}} \in [3, 13] \times [0, 12]$ is used for the results. A Courant-Friedrichs-Lewy (CFL) number of 0.2 was used to ensure stability of the Adams-Bashforth 2nd order temporal scheme, and only every 200th sample was kept for analysis, due to the small timestep used, so each snapshot is spaced $\Delta t = 1.39$ time units apart.

For the LAMP results on this dataset, *one* simulation run is used – the performance does not improve on the addition of more data. It consists of 8,000 snapshots (post-transience), with the first 75% and last 20% used for training and validation respectively. For the ViT-MAE results, an additional seven simulations were run in parallel to provide more data and improve the reconstruction accuracy. All eight runs were seeded with Gaussian noise (of amplitude 0.01) and the chaotic Mean Kinetic Energy (MKE) time series are observed to quickly diverge, so with the transient response removed, all datasets are on independent trajectories from each other. The result is eight independent runs, of which seven are used to train the ViT-MAE, one for validation and one test dataset.

3.2.3 Dataset 3 - Porous Plate LES Wake

This final dataset represents a significant jump in complexity, containing LES data of the wake behind a highly porous flat plate at $Re = 6,000$ (calculated from the plate width and freestream velocity). The dataset contains just 5,000 snapshots with a time step of $\Delta t = 0.084s$. For more details about the data, including details of the experimental PIV setup, see [36].

3.2.4 Data Augmentation

Data augmentation is the process of creating more training snapshots, which resemble the original training data, from a fixed number of samples. This allows the models to train for longer before overfitting. With ViTs' lack of inductive bias, they are known to require significantly more data than CNN-based architectures. This makes data augmentation essential, especially for Dataset 3, which was not only the most complex flow, but also comprised of $< 10\%$ the number of snapshots as Dataset 2. Data augmentation is turned off when calculating the validation loss, to ensure that any change to the distribution from augmenting the data does not affect the results.

Vertical flipping is the simplest technique used and is valid for all three datasets, since the setup of the wakes is symmetric about the line $y = 0$; this doubled the amount of data. Horizontal flipping was not used since all the flows moved in just one direction: left to right. Adding noise to the training was also experimented with, although this was just found to improve robustness to noisy data, instead of delaying overfitting. Another data augmentation technique, used just on Dataset 3, is random cropping, creating by far the most new examples. The available training domain Ω^{train} (of shape $W^{\text{train}} \times H^{\text{train}}$) has already been obtained and the validation domain Ω^{val} (of shape $W \times H$), is a subset of this. The cropped domain Ω_c , also of shape $W \times H$, is uniformly sampled from Ω^{train} each time a training sample is fed into the model.

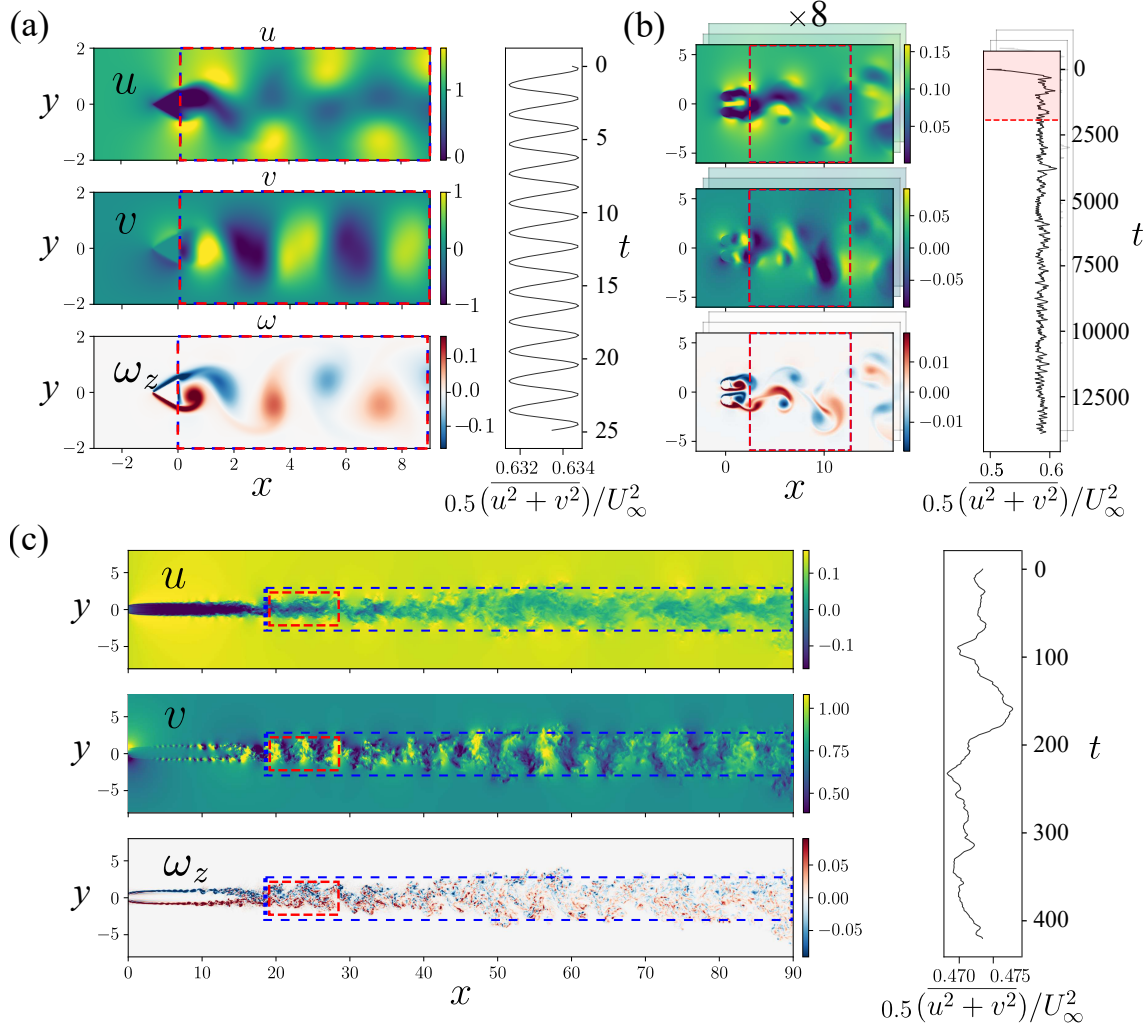


Figure 3.2: Summary of the three datasets. (a) Dataset 1: laminar wake over a triangular bluff body with $Re = 100$, (b) Dataset 2: chaotic two-cylinder wake with $Re = 200$ and (c) Dataset 3: LES simulation of a turbulent wake behind a porous flat plate with $Re = 6,000$. For each, the two velocity components u, v and vorticity w_z are shown. The dashed rectangles represent the training and validation domains. For Datasets 1 and 2, $\Omega^{\text{val}} = \Omega^{\text{train}}$, and for Dataset 3, $\Omega^{\text{val}} \subset \Omega^{\text{train}}$ (coloured red and blue respectively), as discussed in Section 4.3.3. Datasets 1 and 3 each use a single time series (with an interval removed between the training and validation snapshots), whereas Dataset 2 contains eight independent time series, each seeded with random noise, of which seven are used for training, one for validation and one for testing.

$$\begin{aligned}
\Omega^{\text{crop}}, \Omega^{\text{val}} &\subset \Omega^{\text{train}} \subset \Omega, \\
\Omega^{\text{crop}} &= [x^{\text{crop}}, x^{\text{crop}} + W] \times [y^{\text{crop}}, y^{\text{crop}} + H], \\
x^{\text{crop}} &\sim \mathcal{U}\{0, \dots, W^{\text{train}} - W - 1\}, \\
y^{\text{crop}} &\sim \mathcal{U}\{0, \dots, H^{\text{train}} - H - 1\},
\end{aligned} \tag{3.11}$$

where $x^{\text{crop}}, y^{\text{crop}}$ are measured relative to Ω^{train} . Although the flow statistics are noticeably different for snapshots shifted both vertically and horizontally, there can still be enough overlap in what is learned for it to be beneficial for the models' training, as shown in the Results section 4.3.3.

3.3 Training

Training of LAMP models just involves performing POD on the patched training data, followed by two sets of regression, and there are no architectural decisions to make other than the patch size and embedding dimension. Training of the ViT-MAE involved more decisions and each training run was more time consuming, so it was important to be methodical, starting with recommended values from the literature, refining them on Dataset 2 and then using that as a reference for Dataset 3.

3.3.1 Experimentation Framework

In order to streamline the experimentation, all experimentation was conducted in a framework that was created for this project (see Appendix A for details), which promoted three main principles:

1. **Repeatability:** the details of each run (including model architecture, data sources, training parameters) are all stored in a config (configuration) file; at the start of each run, a unique run directory containing this config file is created automatically.
2. **Observability:** the status of the run is continuously reported in a text file, as well as training progress (such as loss curves and an example reconstruction from the best model)
3. **Continuability:** the model state, plus that of the optimiser and loss scheduler is saved every time a new lowest validation loss is achieved.

This made experimentation a smooth process, with the benefits including being able to i) observe the progress of multiple jobs running in parallel ii) cut runs short where progress had stalled iii) keep a record of all previous jobs iv) load, freeze and fine-tune previously trained models.

3.3.2 Hyperparameter Tuning

The architectural parameters (e.g. number of layers, residual dimension etc.) used in [25] were used as a baseline, though the model was shrunk down to account for the smaller dataset. The patch size was the first to be investigated and once the optimal P was determined, that was used for the remaining hyperparameter sweeps (Table 4.2). The 'reference' architecture and training hyperparameters used after this process are given in Tables 3.1 and 3.2, and these values

are used going forwards for all tests on Dataset 2, except where otherwise stated. On Dataset 3, ViT-MAE’s performance was even less sensitive to hyperparameter tuning, where the small number of samples more significantly restricted performance. This is discussed in Section 4.3.3.

Table 3.1: Default model hyperparameters for the ViT-MAE on Dataset 2, obtained from scaling down the architecture in [25]. The model contains a total of 32,551,778 trainable parameters.

	config	value
general	patch size	24
	dropout	0.0
encoder	no. of layers	12
	no. of heads	6
	head dim	64
	FC dim	1536
decoder	no. of layers	4
	no. of heads	4
	head dim	64
	FC dim	1024

Table 3.2: Default training hyperparameters for ViT-MAEs on Dataset 2. Values in [25] were adjusted for the smaller training set.

config	value
fraction unmasked	0.25
optimiser	AdamW
scheduler	cosine annealing
learning rate (start)	5e-4
learning rate (end)	1e-5
weight decay	0.2
epochs	700
batch size	128

3.3.3 Training Schedule

On Dataset 2, a 700-epoch training schedule was sufficient to ensure convergence of the validation loss, where MSE over the reconstructed patches is used as the loss function. The model is optimised with the AdamW [37] optimiser, the de facto standard for training ViTs [14, 25], and the cosine annealing scheduler [38] smoothly decays the learning-rate from its initial value to a minimum over the length of training. Since the loss function does not penalise reconstruction of the unmasked patches (those input into the model), the reconstruction there is poor. The field is known there, however, so for plots of the reconstructed fields, the input data is plotted in unmasked patches. In each input training sample, a constant fraction, f^{unmasked} , of uniformly distributed patches are unmasked. Randomly arranging the patches, increases the variation in the training data, delaying over-fitting.

3.3.4 Artefact Removal

As a consequence of the patching process, the reconstructed u and v fields have a grid-like artefact. As observed on Dataset 1, this problem can in theory be alleviated by reducing the reconstruction error, however on Dataset 2, even on snapshots with good vortex capturing, the artefacts are still noticeable. This is especially true when visualising the (2D) vorticity, $\omega_z = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y}$, since the derivatives highlight the discontinuity at the patch boundaries.

To improve the reconstruction, preliminary experiments were run, adding a residual U-net [39] block to the end of the ViT-MAE, importantly after the patch unembedding block, so it operated on the full $H \times W$ flow-field, instead of $P \times P$ patches. This enabled it to smooth out the solution at the boundaries. An additional physics-informed loss term, acting on the reconstructed

vorticity field and scaled by factor $\beta_\omega \in \mathbb{R}$ was investigated to specifically tackle the issue of high error on the patch boundaries.

$$\mathcal{L}_{\omega_z}^{\text{pred}} = \|\tilde{\mathbf{X}}^\star - \mathbf{X}^{\text{test}}\|_2^2 + \beta_\omega \|\omega_z(\tilde{\mathbf{X}}^\star) - \omega_z(\mathbf{X}^{\text{test}})\|_2^2, \quad (3.12)$$

where $\omega_z(\cdot)$ uses a second order central difference to approximate the vorticity field. Loss terms acting on the gradients of the reconstructed field have proven to be successful in studies on super-resolution [40] and in general, physics-informed loss functions [41] can target specific physical condition that should be observed (e.g. continuity and temporal consistency between adjacent snapshots).

Two training approaches were tested, one in which the whole architecture (ViT-MAE plus a U-Net) is trained from scratch and the other in which the ViT-MAE is trained to sufficient loss (without a U-Net), its weights are frozen and then a U-Net is trained on top of it. A U-Net-style encoder-decoder [39] is used with two modifications: same-padding convolutions are used to preserve the spatial dimensions between layers and it is used as a residual block with the weights of the final convolutional layer initialised to zero - this eases training, ensuring that the U-Net only needs to learn the *correction*. Results can be found in Section 4.2.7.

3.4 Evaluation

With models trained on all three datasets, their performance was evaluated across a range of tests to investigate their utility in engineering applications.

3.4.1 Reconstruction Mean Squared Error

The first metric to test is the MSE of reconstructed snapshots – if this error is low, the model is expected to perform well on all other tasks. MSE is defined as in Eqn. 3.8 and for field plots of the instantaneous error, the value of error plotted is $\tilde{\mathbf{X}}^\star - \mathbf{X}^{\text{test}}$ – this signed error shows where the model over- vs under-predicts.

For Dataset 1, a random array of uniformly distributed patches covering just 10% of the domain is used as the input data, mirroring the setup in [25]. Each loss value quoted is the *median* value across 25 random arrangements – the mean is not used because, especially with the noisy reconstructions, certain random patch arrangements are particularly poorly placed and a single high loss value from this can significantly skew the mean.

For the results on Dataset 2, 25% of the field is input into the model, the patches forming a single rectangle spanning the entire height of the domain and from $0.25W$ to $0.5W$ in the x -direction. This ensures a fair test and more closely matches an actual experimental setup. On Dataset 3, the reconstruction results are presented using uniformly-spaced columns of height H and varying widths to demonstrate how the reconstruction accuracy degrades as the distance from the input windows increases.

3.4.2 Noise Removal

To test their robustness, LAMP and ViT-MAE were then tested on noisy data. To simulate noise introduced by a sensor, Gaussian noise is added to the input data, with the power relative to the input, unnormalised data.

$$\hat{\mathbf{X}}^{\text{noisy}} = \hat{\mathbf{X}}^{\text{masked}} + \boldsymbol{\varepsilon}, \quad \varepsilon_i \sim \mathcal{N}(0, P_{\text{noise}}), \quad P_{\text{noise}} = \mathbb{E} \left[\left(\hat{\mathbf{X}}^{\text{masked}} \right)^2 \right] 10^{-\text{SNR}/10}, \quad (3.13)$$

Values of $\text{SNR} \in [10, 40]$ are tested to inform how the models should perform on PIV data. Typical PIV uncertainty lies in the range of approximately $\pm 0.025 U_\infty$ at 95% confidence for planar velocity measurements [42], corresponding to $\text{SNR} \approx 30$ dB, so the target is to achieve good reconstruction accuracy at this level of noise.

3.4.3 Distribution Plots

For some downstream tasks, reconstructing the exact flow-field at each instant is less important than capturing distributions along key slices, the flow statistics, or the frequency spectrum of a reconstructed time series. These also provide a more detailed description of the reconstruction quality than the MSE, a single scalar.

For Datasets 1 and 2, profiles of the Root Mean Squared (RMS) velocity components u_{rms}, v_{rms} (normalised by the free-stream velocity U_∞) are calculated against the wall-normal direction y , along the line $x/W = 0.625$. The normalised Reynolds shear stress $u'v'/U_\infty$ is also calculated along this same line. Plotting the normalised mean stream-wise velocity $\bar{u}/U_\infty$ along the line $y = 0$ shows the centreline recovery. In addition to the stream-wise and wall normal distributions, probability density functions for velocity perturbations u', v' at $y = 0, x/W = 0.625$ are plotted, as well as the power spectral density of the two velocity perturbations at the same location.

4 - Results and Discussion

4.1 Dataset 1 (LAMP vs Gappy POD)

This section presents the results of LAMP on Dataset 1, alongside Gappy POD for comparison, to understand how LAMP performs at reconstructing a periodic and spatially coherent (i.e. ‘global’) flow-field. LAMP can reconstruct the flow almost flawlessly on clean data, and simultaneously reconstruct and filter noisy inputs. Due to the spatial coherence of the flow, it is shown that subdividing the domain into small patches is unnecessary and detrimental to the performance.

4.1.1 POD Reconstruction Loss

Curves of patch-wise POD reconstruction error ($\mathcal{L}^{\text{POD}}$) are plotted in Fig. 4.1 for different choices of patch size and embedding dimension. The global POD basis (used by Gappy POD) is the special case where a single rectangular patch covers the entire domain, and is added to the figure as a baseline.

As seen in Fig. 4.1 (a), the vast majority of the energy is captured by just the first few POD modes, which is consistent with what is commonly seen in the literature for periodic wakes [15, 35]. The compression factor is defined as the efficiency of compressing d_P -dimensional patches into a d -dimensions latent representations. Fig. 4.1 (b) shows that the same $\mathcal{L}^{\text{POD}}$ is achieved by larger patch sizes with much higher compression factors, which is expected for such a global flow-field – simple patch-wise POD does not benefit from subdividing the coherent patterns present in a Von Kármán vortex street [43]. For a comparison, Figs. 4.8 and 4.18 show what these profiles look like for chaotic and spatially-localised flows.

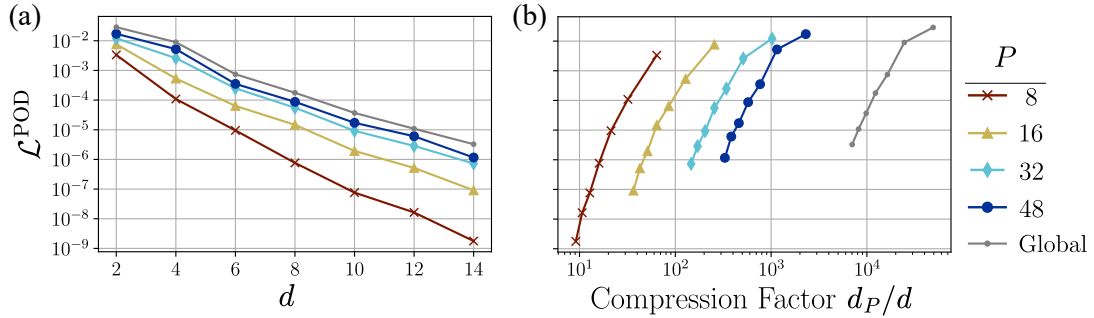


Figure 4.1: Patch-wise POD reconstruction error for varying patch size P and embedding dimension d . Autoencoder loss $\mathcal{L}^{\text{POD}}$ is plotted against (a) latent dimension, d and (b) compression factor, d_P/d . The grey line shows the reconstruction accuracy of a global POD basis.

4.1.2 Masked Reconstruction

Having trained an array of LAMP models, with a sweep of patch sizes and embedding dimensions, and using them to reconstruct the flow-field from 90%-masked inputs, one can observe clear trends (see Fig. 4.2). For all four patch sizes, the loss curves decrease with increasing d , at similar rates, until they start to plateau, with this occurring at lower d values for smaller patches. As one can see from the reconstructed field (Fig. 4.2), distributions (Fig. 4.3), proba-

bility density function (Fig. 4.4) and the power spectrum density (Fig. 4.5), LAMP is able to reconstruct both the instantaneous flow-field and its dynamics flawlessly. Appendix C shows the R^2 values of the different reconstructed distributions, for which $R^2 = 1.000$ for all eight distributions.

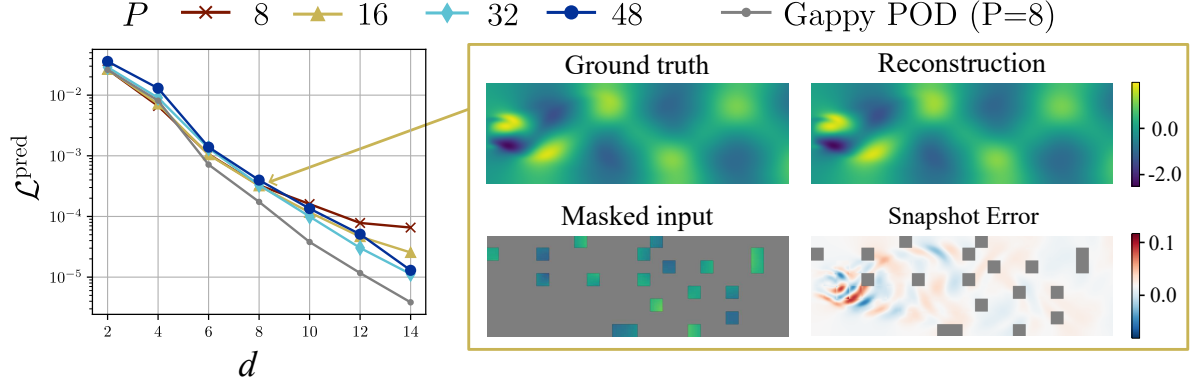


Figure 4.2: Reconstruction error, $\mathcal{L}^{\text{pred}}$, is presented for a range of patch sizes and embedding dimensions, calculated from the median of 25 random patch arrangements, and $f^{\text{unmasked}} = 0.1$ is used. On the right is an example reconstruction with $P = 16$, $d = 8$, demonstrating that the reconstruction is indistinguishable from the ground truth by eye with very few modes.

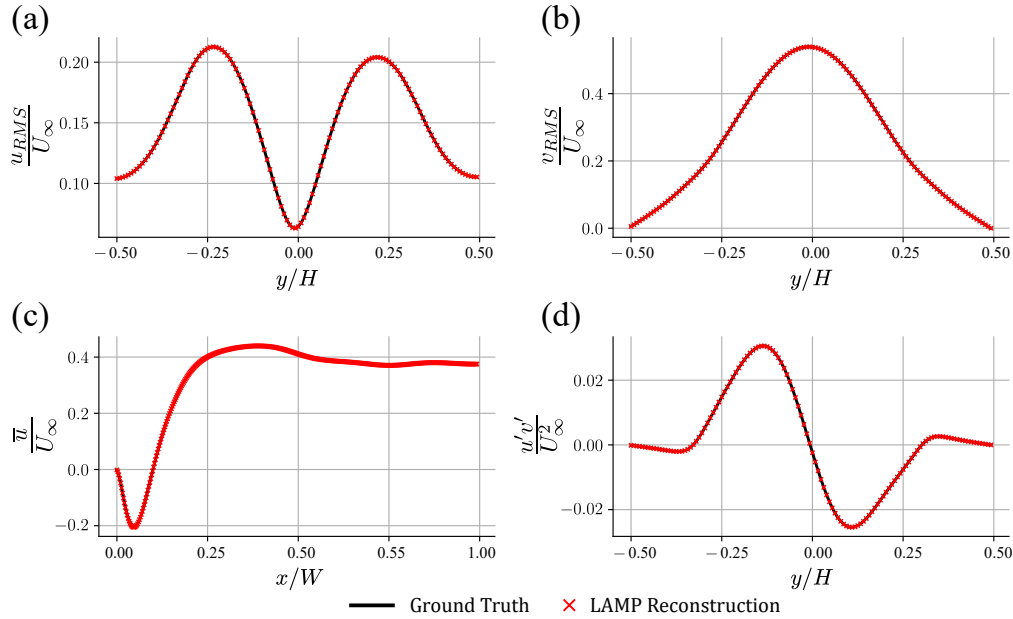


Figure 4.3: Key velocity distributions along the lines $x/W = 0.625$ (a, b, d) and the centreline, $y = 0$ (c), for flow-field reconstructed by the LAMP model which achieved the lowest $\mathcal{L}^{\text{pred}}$ (with $P = 48$, $d = 24$). The normalised (a) RMS u , (b) RMS v , (c) centreline recovery and (d) Reynolds shear stress distributions are all plotted and $R^2 = 1$ to within numerical precision for all four distributions, showing a perfect reconstruction.

4.1.3 Noise Removal

The results of adding noise to Dataset 1 are shown below in Fig. 4.6, and demonstrate that LAMP is robust to the addition of noise – $\mathcal{L}^{\text{pred}}$ is consistently lower than the noise floor of the input

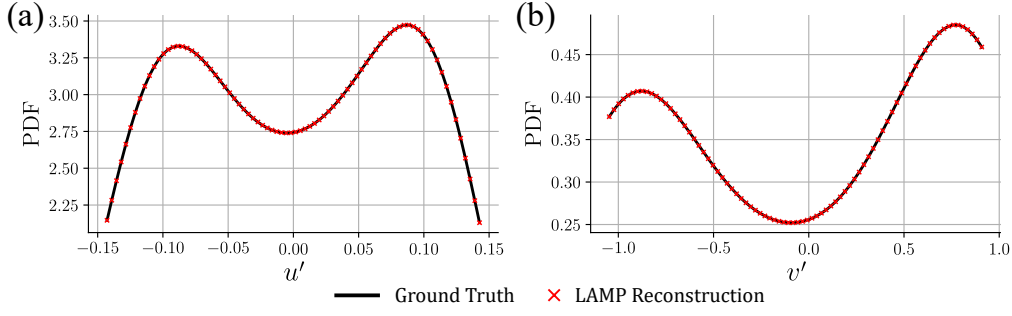


Figure 4.4: The probability density function for each velocity perturbation component (a) u' and (b) v' is calculated at the point $x/W = 0.625$ on the centreline. Again, the distribution for the reconstructed field exactly matches the ground truth. The reason for the asymmetry in the ground truth v' distribution is simply because the number of snapshots tested is not a multiple of the oscillation period.

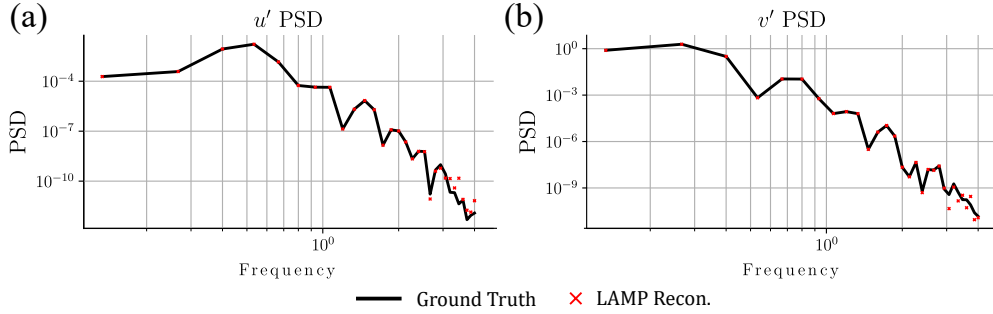


Figure 4.5: The power spectral density for (a) u' and (b) v' is calculated, again at $x/W = 0.625, y = 0$. (a) shows slight disagreement in the PSDs at the highest frequencies, and in (b) this discrepancy is even smaller. Accurate capturing of the PSD shows that LAMP can accurately capture the dynamics of the flows it reconstructs.

data – although this only holds for sufficiently large patch sizes. The ability to simultaneously reconstruct and de-noise is a clear demonstration of the power of ROM. The effect of increased noise on the loss curves (with $P = 32, 48$) is to shift the $\mathcal{L}^{\text{pred}}$ minima towards lower values of d , and this reflects the corruption of later modes shown in Fig. 2.1 (c). For smaller patch sizes however (namely $P = 8, 16$), $\mathcal{L}^{\text{pred}}$ increases significantly – with a lower compression factor, POD is less able to separate signal from noise, resulting in spurious correlations. Gappy POD is more robust to noise, which is consistent with the trend of increasing patch sizes performing better – its *global* POD basis leaves the greatest separation between signal and noise in the latent space, enabling the most efficient noise filtering.

4.1.4 Attention Maps

Next, we focus on the attention matrix, calculated as a by-product, yielding an interpretable method for informing sensor placement. By taking the columnwise mean of the log-attention matrix, $\mathbf{A}$, we obtain the predictive power of each patch – how well each patch predicts the rest of the domain – and by reshaping this back into the physical space, we obtain an interpretable ‘attention map’ of optimal single-sensor placement (see Fig. 4.7).

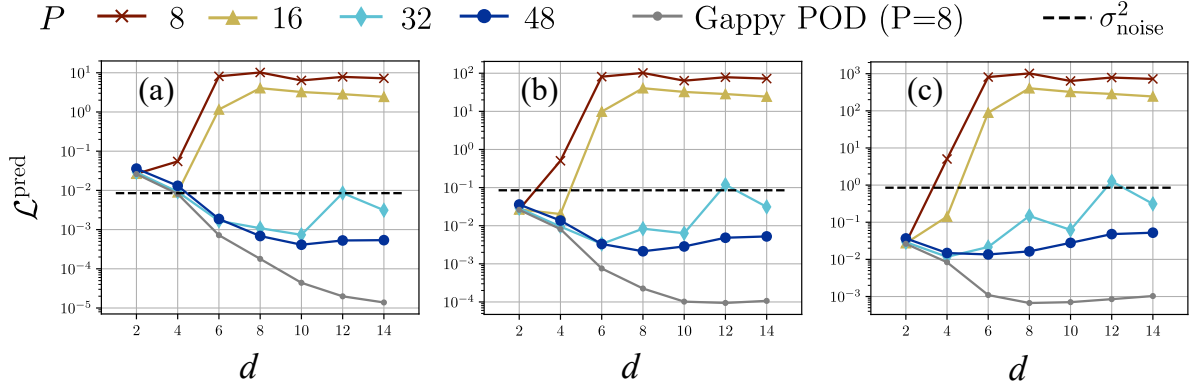


Figure 4.6: Reconstruction error $\mathcal{L}^{\text{pred}}$ is plotted for varying noise levels, patch sizes P and latent dimensions d , all with 10% patches unmasked. The input is noisy masked data with $SNR =$ (b) 30 dB, (c) 20 dB, (d) 10 dB. The loss is computed against the noise-free target $\mathbf{X}^{\text{test}}$ and the model is trained on noise-free data. Horizontal dashed lines show the noise variance.

Increasing P , the resolution of the attention maps naturally decreases and the range of values also reduces, however the position of the attention peaks remains largely the same. By contrast, the positions of the attention peaks vary significantly with d , particularly from $d = 4$ to 8. As d increases, high-power regions multiply from one or two near-body patches to distributed coverage across shear layers and the far wake, which mirrors the underlying POD hierarchy. Section 5.3 discusses how these attention maps could inform optimal sensor placement.

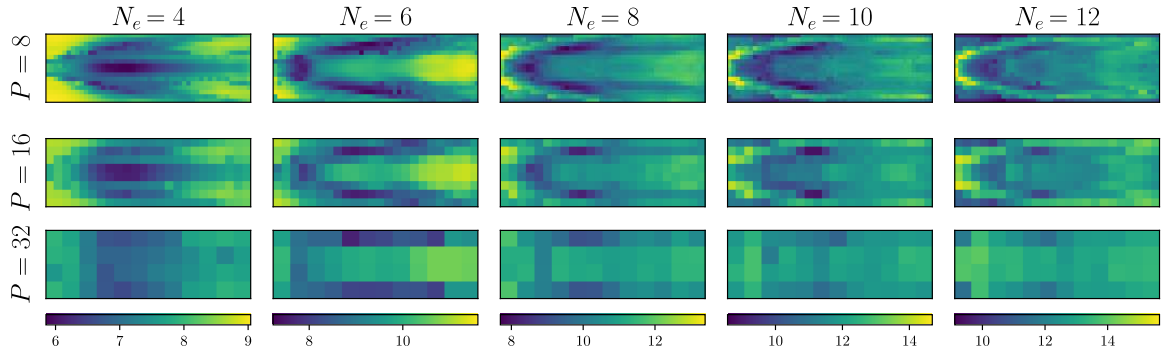


Figure 4.7: Attention maps for varying P and d , calculated taking a mean over columns of the log-attention matrix $\mathbf{A}$. The latent flow state in brighter regions is able to, on average, predict the rest of the patches with lower loss. This offers a data-driven strategy for optimal multi-fidelity sensor placement.

4.2 Dataset 2 (LAMP, Gappy POD and ViT-MAE)

The same analysis is repeated on Dataset 2 for both LAMP and Gappy POD and the results show a significant decrease in performance. The LAMP reconstructions lack important details of the downstream flow structures, whereas Gappy POD over-confidently predicts flow in these regions. To tackle this, the ViT-MAE is tested, which is able to achieve an order of magnitude lower loss and predicts key flow distributions with R^2 values much closer to 1.

4.2.1 POD Reconstruction Loss

To analyse the flow complexity, the patchwise POD reconstruction loss, $\mathcal{L}^{\text{POD}}$, is again calculated and shown below (Fig. 4.8), for contrast with Dataset 1 (Fig. 4.1). In Fig. 4.8 (a), one can observe a more gradual decrease in loss than with Dataset 1 – the energy is spread across more modes without such a sharp cut-off. With chaotic flows, the state-space trajectory explores a much larger and high-dimensional latent space, so a linear latent representation is consequently less compact. Additionally, chaotic wakes contain intermittent events, and since POD seeks to optimise the *average* reconstruction, these require many more modes to capture. In the case of the two-cylinder flow, these intermittent events include consecutive same-side vortex shedding, which break the typical alternating antisymmetric pattern. Finally, the flow behind the two cylinders is less stable, with the two wakes interacting and disrupting the regular Kármán shedding. As explored in Fig 4.12, two separate snapshots with almost identical flow structures upstream can have entirely different states downstream.

Due to the shorter-range correlation and reduced overlap in the POD representations of different patches, the curves in Fig. 4.8 (b) are shallower. A global POD basis, which correlates flow across the length of the domain, is thus less suitable for this dataset.

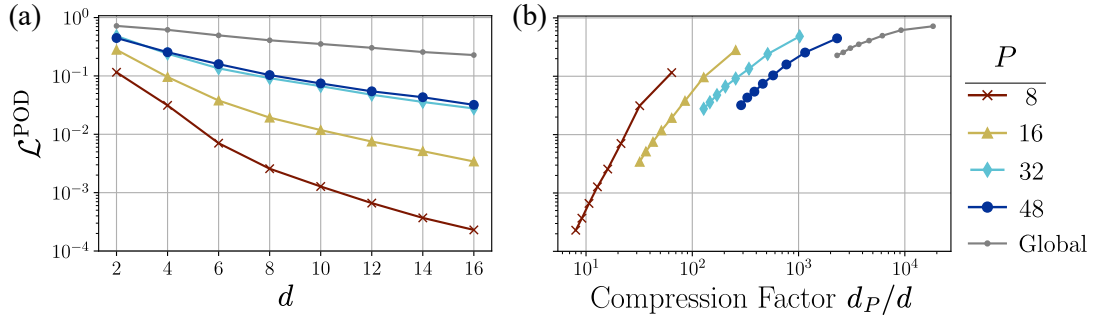


Figure 4.8: Autoencoder loss is plotted against the (a) embedding dimension and (b) compression factor. The loss descends less steeply with increasing d than with Dataset 1.

4.2.2 Masked Reconstruction (LAMP vs Gappy POD)

LAMP and Gappy POD are then tasked with reconstructing snapshots of the two-cylinder flow from a single rectangular window covering 25% of the domain. This provides a fair comparison – all values of P reconstruct the same input data – and resembles an experimental setup with PIV input data.

Observing the loss curves in Fig. 4.9, $\mathcal{L}^{\text{pred}}$ plateaus at significantly higher values than on Dataset 1 and this plateau occurs at later values of d . Reconstructions of an example snapshot, and the corresponding error plots, demonstrate *how* the two methods differ in their error: LAMP takes a weighted average of the predictions from all unmasked patches, so tends to average out the predictions where there is high uncertainty. This results in overly-diffuse reconstruction fields far from the unmasked window. Note that the vortex in the bottom right is neglected entirely. On the other hand, Gappy POD extrapolates global POD modes that reconstruct the field well *within* the PIV window to the rest of the domain with full confidence. This leads to massively over-predicting the flow structures far downstream and upstream. As seen in Fig. 4.9 (b), this gets worse, not better, with the addition of more modes, as Gappy POD finds more spurious

correlations. The attention mechanism in LAMP is able to attenuate these spurious correlations, creating a stable decrease in loss. The plots in Figs. 4.15 to 4.17 show more clearly the deficiency of linear methods for predicting the distributions describing the flow. Extending LAMP with *multi-layer*, *multi-headed* transformers is required to predict nonlinear relationships between patches.

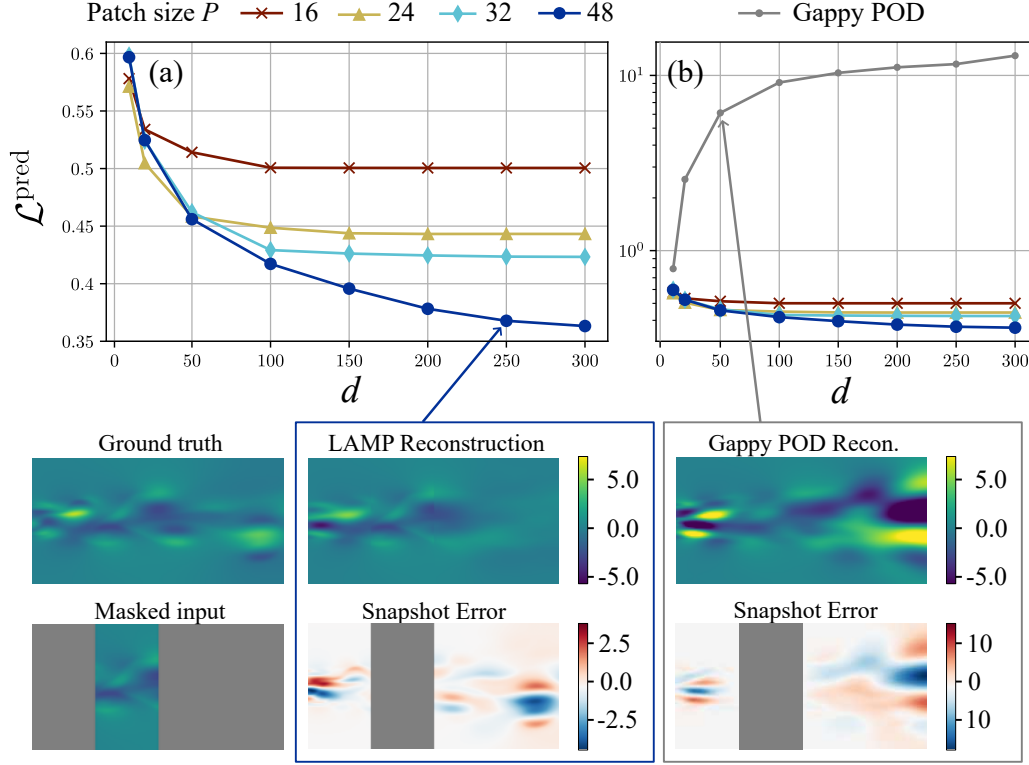


Figure 4.9: Masked reconstruction of the chaotic wake from 25% unmasked patches, all located within the region $0.25 < x/W < 0.5$. The reconstruction of one snapshot with the best LAMP model ($P = 48, d = 250$) is shown below. The Gappy POD reconstruction shown is not from the lowest-loss model, to illustrate the difference between the tentative LAMP and confident Gappy POD predictions. The same qualitative behaviour is observed throughout the range of latent dimensions tested.

4.2.3 LAMP Attention Map

Observing the attention maps generated with a wide range of d , there is a much smaller difference between them, both in the qualitative appearance and magnitude of log-attention. This saturation in appearance mirrors the more slowly decaying $\mathcal{L}^{\text{pred}}$ vs d curves. All three attention maps show similar attention peaks along the top and bottom of the domain, a stark difference from Dataset 1 – with the linear attention mechanism of LAMP unable to reconstruct significant details in the flow, the attention map is able to provide a much less rich picture on optimal sensor placement. This also shows the limitation of using the LAMP’s attention maps: they are no longer informative if the model performs poorly. This motivates analysis of the attention maps present within ViTs.

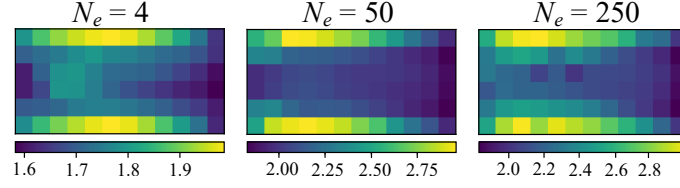


Figure 4.10: Attention maps for $P = 16$. Values range from 1.6 to only 3.0 from $d = 4$ to 250 compared to 6 to 15 from $d = 6$ to 12 in Fig 4.7. This represents a much higher uncertainty (the log-attention plotted is the average *negative* log error of pair-wise linear predictions).

4.2.4 ViT-MAE Hyperparameter Tuning

With LAMP and Gappy POD both generating high-loss reconstructions, the Vision Transformer Masked Autoencoders are applied to the two-cylinder wake dataset. Unlike the other two models, there are numerous parameters to tune, (see Section 3.3.2) both in the architecture and training, before any analysis can be done on the reconstructed flow-fields.

Patch size was investigated first, since it fundamentally changes the input and output of the model, and Table 4.1 shows that $P = 24$ is the optimal patch size. Although results depend on random weight initialisation, the table shows that this variation between runs is of an acceptable level and appears to decrease as $\mathcal{L}^{\text{pred}}$ decreases. Fig. 4.11 shows an example snapshot reconstructed by this ViT-MAE, showing almost an order of magnitude lower $\mathcal{L}^{\text{pred}}$ than LAMP, with far less diffuse predictions downstream.

Table 4.1: $\mathcal{L}^{\text{pred}}$ for a range of patch sizes, identifying $P = 24$ as the optimal value. The maximum and minimum values across three runs are shown, giving an indication of the spread of results.

P	min. $\mathcal{L}^{\text{pred}}$	max. $\mathcal{L}^{\text{pred}}$
12	0.142	0.159
16	0.098	0.105
24	0.058	0.059
32	0.059	0.063
48	0.065	0.069

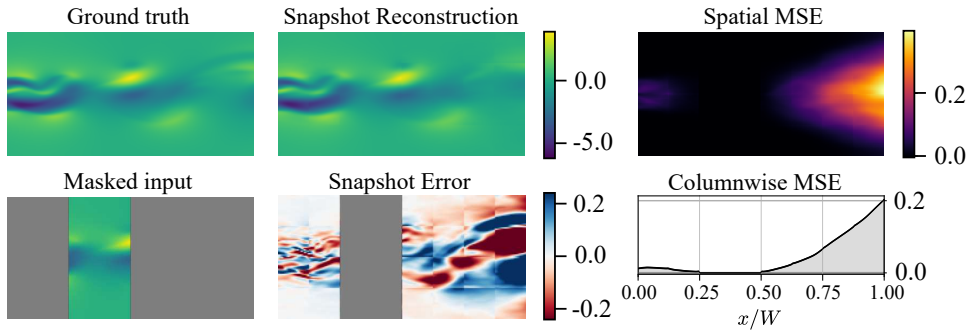


Figure 4.11: An example flow-field reconstructed by a ViT-MAE with $P = 24$, all other parameters given in Table 3.1, showing a significant improvement on LAMP and Gappy POD (Fig. 4.9)

With the optimal patch size $P = 24$ determined, parameter sweeps were then conducted over nine hyperparameters (see Table 4.2) to show the sensitivity of $\mathcal{L}^{\text{pred}}$ to key architectural and training variables to guide future model improvements. As shown in this table, the encoder transformer blocks are generally undersized, whilst the decoder is closer to optimal. That the best performance is obtained with a larger encoder than decoder agrees with what is expected

from the literature [25]. Regarding the other training parameters, all levels of additional training noise tested harmed training when testing on clean data (although Section 4.2.6 shows the utility of training noise), and only a minor amount of dropout (0.05) was helpful. Also as expected from the literature, the loss shows no sign of plateauing with the addition of more data.

No further optimisations were performed on the models for Dataset 2 – the reference architecture is used for the remaining ViT-MAE results – but the parameter sweep informed the order in which the Dataset 3 models were tuned. More important than finding a local minimum across all parameters is to understand how and where the model fails, as this illustrates the reliability of the model and can inform entirely new avenues of discovery – this is discussed in the next section.

Table 4.2: Results for a parameter sweep over the width, N_h , and depth, N_l , of the attention blocks, width of the MLP, d_{MLP} , power of training noise, SNR, degree of dropout and number of samples. The **grey** refers to the default settings and **bold** highlights the lowest loss per table.

(4.2.1) Encoder depth: deeper encoder is better		(4.2.2) Encoder width: much wider than decoder is optimal		(4.2.3) Encoder MLP width: MLP is undersized	
N_l^{enc}	$\mathcal{L}^{pred}$	N_h^{enc}	$\mathcal{L}^{pred}$	d_{MLP}^{enc}	$\mathcal{L}^{pred}$
4	0.061	4	0.059	768	0.63
8	0.059	6	0.058	1024	0.59
12	0.058	8	0.054	1536	0.058
16	0.052	16	0.051	2048	0.054
(4.2.4) Decoder depth: gradual improvement with depth		(4.2.5) Decoder width: 4 is optimal; high sensitivity.		(4.2.6) Encoder MLP width: 1024 is optimal; high sensitivity.	
N_l^{dec}	$\mathcal{L}^{pred}$	N_h^{dec}	$\mathcal{L}^{pred}$	d_{MLP}^{dec}	$\mathcal{L}^{pred}$
2	0.062	2	0.066	512	0.065
4	0.058	4	0.058	1024	0.058
8	0.056	8	0.079	1536	0.065
12	0.055				
(4.2.7) Training noise: harms performance on clean test data.		(4.2.8) Dropout: helps performance in moderation.		(4.2.9) Training snapshots: no plateau yet observed	
SNR (dB)	$\mathcal{L}^{pred}$	dropout	$\mathcal{L}^{pred}$	λ	$\mathcal{L}^{pred}$
∞	0.058	0	0.058	8,000	0.130
30	0.066	0.05	0.53	24,000	0.082
20	0.074	0.1	0.59	32,000	0.070
10	0.092			40,000	0.061
				48,000	0.058

4.2.5 ViT-MAE Time Series and Anomaly Analysis

All predictions are made on a single frame at a time without any time history, however since the input data is a time series, we can reconstruct successive frames to track the error. The first thing to note in Fig. 4.12 (a) is that $\mathcal{L}^{pred}$ varies continuously (as opposed to unstructured noise). It was observed that the vortex structure from neighbouring snapshots *generally* exhibit temporal consistency (except at distinct error spikes) and this is important for any downstream

tasks which involve approximating time derivatives from the reconstructed fields. The second thing to note is that there are occasional but severe loss spikes, which causes the distribution to be approximately log-normal – positively skewed with a long rightward tail (see Fig. 4.13). Two hypotheses were tested to explain these loss spikes.

The initial hypothesis was that the error spikes are caused by the input data at the spikes existing outside the training distribution, however it was not substantiated: The mean kinetic energy and mean squared vorticity of the test dataset, and their time derivatives, were regressed against $\mathcal{L}^{\text{pred}}(t)$, but no statistically significant correlation was found. The internal signal generated from applying the model’s patch embedding to the input data was also analysed: the Mahalanobis distance [44] of this was calculated with respect to the distribution of the training data (see 4.12 (a)), however regressing against $\mathcal{L}^{\text{pred}}$ gave an R^2 value of just 8×10^{-4} .

The second hypothesis was that the ill-posedness of the static reconstruction task causes the spikes. Instead of looking at the distributions of various metrics, an explanation for the loss spikes came from searching within the training data for snapshots with a similar flow state within the input window at these spikes. This search involved:

1. conducting POD analysis on the training data, using just the flow state in unmasked region
2. projecting the same region of test data into the POD space
3. searching for the nearest-neighbour training snapshot from a given high-test-loss snapshot (where distance is defined as the Euclidean distance in POD space)

In Fig. 4.12 (b), the concept of nearest POD neighbours is illustrated, plotting just the first two principal components (PCs) and from Fig. 4.12 (c), one can observe the visual similarity between nearest-neighbour pairs *within* the input region. Since the data is spatially (as well as temporally) chaotic, minute changes to the input data can result in completely different vortical structures downstream. This error overshadows the contribution from visual artefacts (namely the patching process), so should be addressed first. A likely solution to this problem is to modify the ViT-MAE to accept a history of *previous* snapshots and is discussed in Section 5.3.

4.2.6 ViT-MAE Noise Filtering

Testing the ViT-MAE on noisy data, it is clear that the results from the SNR sweep (shown in Table 4.2) provide an incomplete picture. Training on noisy data *does* increase the error when testing on clean data, but also increases robustness to noise at test time see (Fig. 4.13). Looking at the 20 and 30 dB curves in (a) and (b), the variance of the distribution decreases significantly as amplitude of the loss spikes are reduced.

It was found that training models on noisy data is more effective than first training on noise-free data and then applying POD-based de-noising at test time, suggesting that both the embedding process *and* the attention mechanism are trained to be robust to noise. Whereas LAMP and Gappy POD automatically have noise-removal in-built, due to the use of POD, this has to be incentivised if trained with gradient descent. Models deployed on noisy data should thus be trained with noise in mind, although noise removal could potentially be introduced through

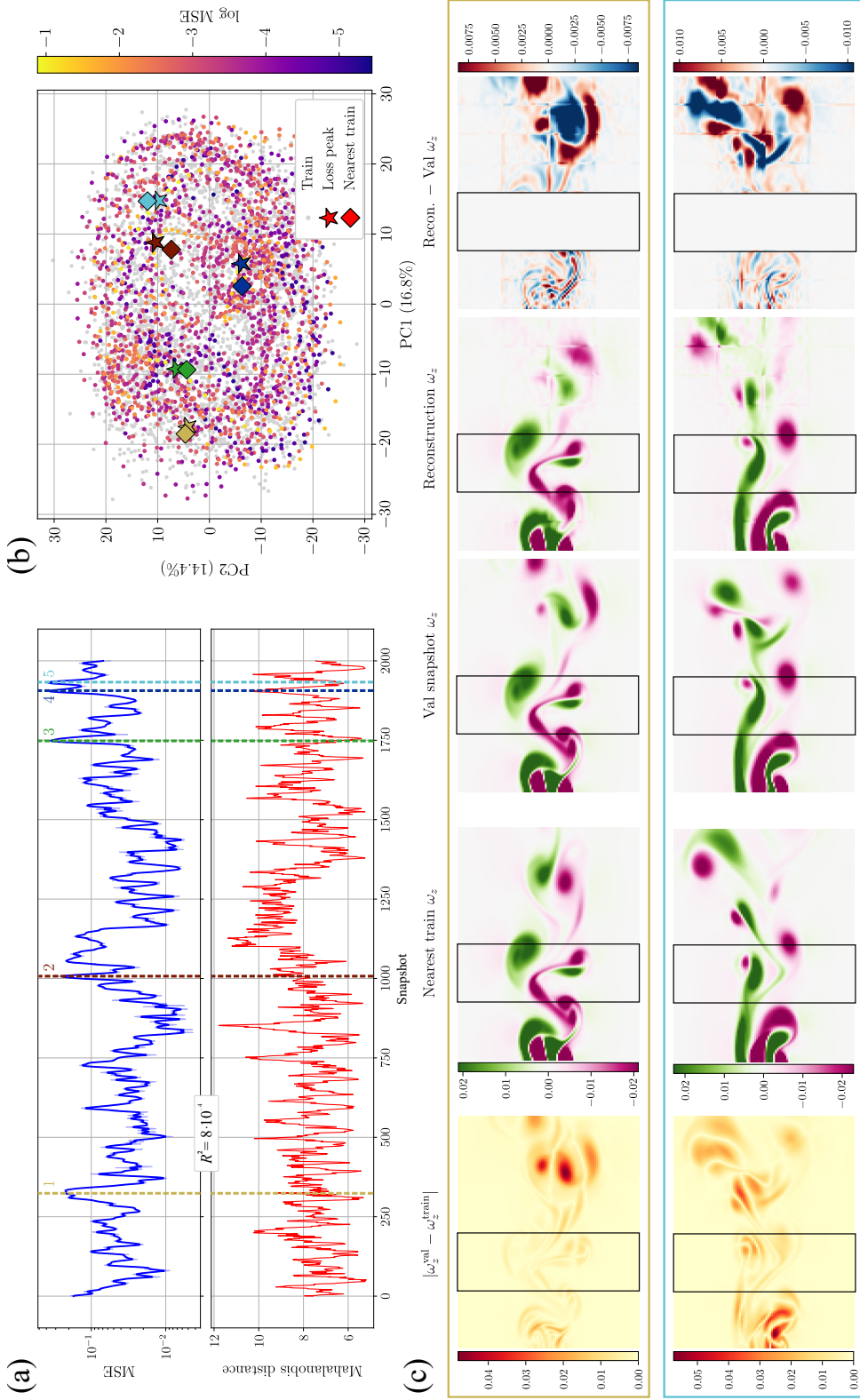


Figure 4.12: Anomaly analysis of the ViT-MAE reconstruction error across all test samples. (a) shows the test MSE (blue) and the Mahalanobis distance of the patch embedding from the centroid of the training embedding (red). The negligible $R^2 = 8 \times 10^{-4}$ value suggests that the loss peaks are not associated with out-of-distributed inputs at the patch embedding state. The five highest local maxima of the $\mathcal{L}^{\text{pred}}$ curve are found and labelled (1 – 5). (b) shows projections of the patch embedding onto the first two principal components (i.e. POD modes) of the training distribution (grey) with validation snapshots coloured by $\log(\mathcal{L}^{\text{pred}})$. Stars indicate the projection of the five loss peaks and the diamonds show their nearest-neighbour training samples in PCA space. Peaks 1 and 5 are used for the snapshot comparison in (c), which shows the following vorticity fields, from left to right: difference between the validation snapshot and its nearest training neighbour, the nearest neighbour, the validation snapshot, ViT-MAE reconstruction of the validation snapshot and reconstruction error. The flow-field in the input window matches closely between the validation snapshot and nearest neighbour, but differs significantly downstream.

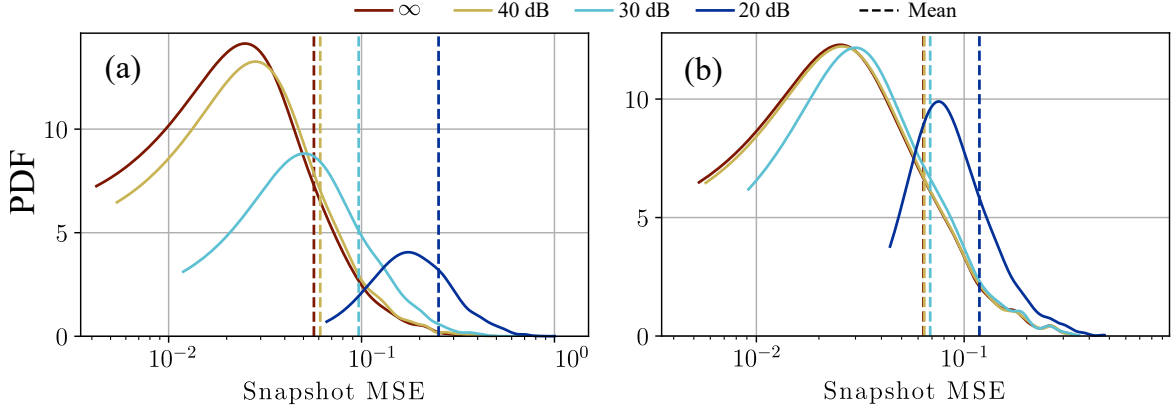


Figure 4.13: Probability distributions of per-snapshot MSE for two ViT-MAEs tested on Dataset 2 with added noise. (a) shows the results of the baseline model (trained on clean data) and (b) uses a model with the same architecture, but trained on noisy 30 dB training data. The model trained on noisy data performs slightly worse on the low noise data, but better on 20 and 30 dB noise. The distributions resemble a log-normal distribution (note that the x-axis is logarithmic).

fine-tuning to save having to train from scratch. Crucially, this technique of training on noisy data reduces the mean error on data with 30 dB noise (the expected experimental noise) by 30% to close to the error on noise-free data – the target of robustness to noisy data has thus been achieved.

4.2.7 Artefact Removal

To test the U-Net’s ability to remove artefacts left over from the patching process, two models were trained, with the reference architecture in Table 3.1 used for the ViT-MAE backbone. The vorticity loss term is added during training to incentivise accurate reconstruction of the vorticity fields and a factor $\beta_\omega = 0.01$ appropriately scales this. Of the two approaches described in Section 3.3.4, it was found that training the ViT-MAE/U-Net combined model from scratch led to a 4% lower loss than training the ViT-MAE backbone first and then adding the U-Net, so the results of the model trained from scratch are presented in Fig. 4.14 (b), compared to the original model (without the U-Net block) in Fig. 4.14 (a). As one can observe, the U-Net successfully removes the grid-like artefacts to the point where they are no longer visible. The increased error observed at the boundary of the input window arises due to the increased loss of the predicted field in the unmasked region, and defining the vorticity in terms of the *ground truth* vorticity field would remove this.

4.2.8 Distributions Comparison

With all three models being tested on Dataset 2, the three models can be compared, firstly in terms of the distributions of the reconstructed flow-fields (Fig. 4.15). The principle observation is how much better the ViT-MAE performs, compared to LAMP and Gappy POD. Looking at the RMS velocity distributions along y/H (Fig. 4.15 (a) and (b)), LAMP performs well towards the edges (towards the freestream). Towards the centre, the shape of the distributions are similar, but it under-predicts the magnitude. On the other hand, Gappy POD captures the overall magnitude better, but the shape is worse. These two observations agree with the

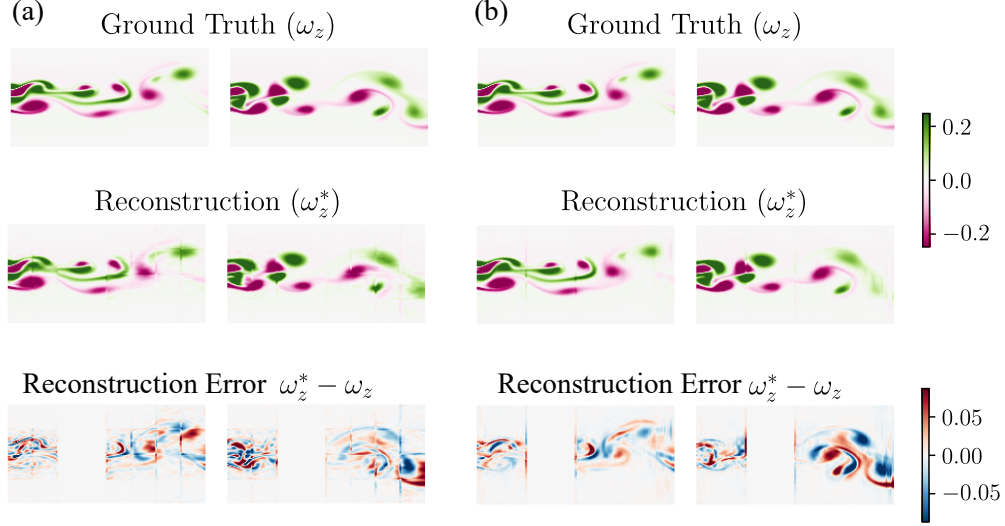


Figure 4.14: The vorticity plots reconstructed by the (a) reference ViT-MAE and (b) ViT-MAE with U-Net are shown. The reconstructed vortices are smoother and the error plot does not contain the grid artefacts.

understanding that LAMP smooths out the solution where uncertainty is high, whilst LAMP extrapolates (often incorrectly) with full confidence. The ViT-MAE balances the best of both of these, producing the correct shape for both, only slightly underestimating the magnitude of the u_{RMS} distribution. All three models perform well in Fig. 4.15 (c), with only Gappy POD slightly under-predicting $\bar{u}$ in the region immediately after the PIV window. It is expected that all models should perform well, since this metric is calculated of the mean flow behaviour. Finally, comparing the Reynolds shear stress, Gappy POD completely fails to capture the shape of the distribution – since this quantity is a product of two reconstructed quantities, the errors accumulate to produce an even more erroneous distribution. LAMP’s predictions are better, but still have significant error compared to ViT-MAE, which manages to capture the shape well, except for a steep gradient at patch boundary $y/H = 0.25$ – this comes as a consequence of making predictions with non-overlapping patches, but is less noticeable at the other boundaries $y/H = 0.5$ and 0.75 .

In Fig. 4.16 (a), the results for LAMP and Gappy POD corroborate what is found in Fig. 4.15 (a): they both under-predict the variance of the velocity fluctuations on the centreline at $x/W = 65\%$, resulting in a narrower PDF and lower u_{RMS} at $y = 0$. On the other hand, ViT-MAE is able to better capture the positive skew of the PDF, although there is still discrepancy between the ViT-MAE prediction and the ground truth. The smoothing of the wake (illustrated in Fig. 4.9) is also seen in Fig. 4.16 (b), with a lower-variance distribution centred at zero. For the v_{RMS} distribution, ViT-MAE matches the distribution much better.

In the power spectral densities of both (a) u' and (b) v' (Fig. 4.17), ViT-MAE maintains high accuracy across the full range of frequencies. In contrast, LAMP and Gappy POD significantly over- and under-predict the power of the highest frequency content. This shows the need for ViT-MAE in applications which require high temporal resolution. The value of R^2 for each distribution is given in Appendix C, but to summarise, LAMP and Gappy POD have similar,

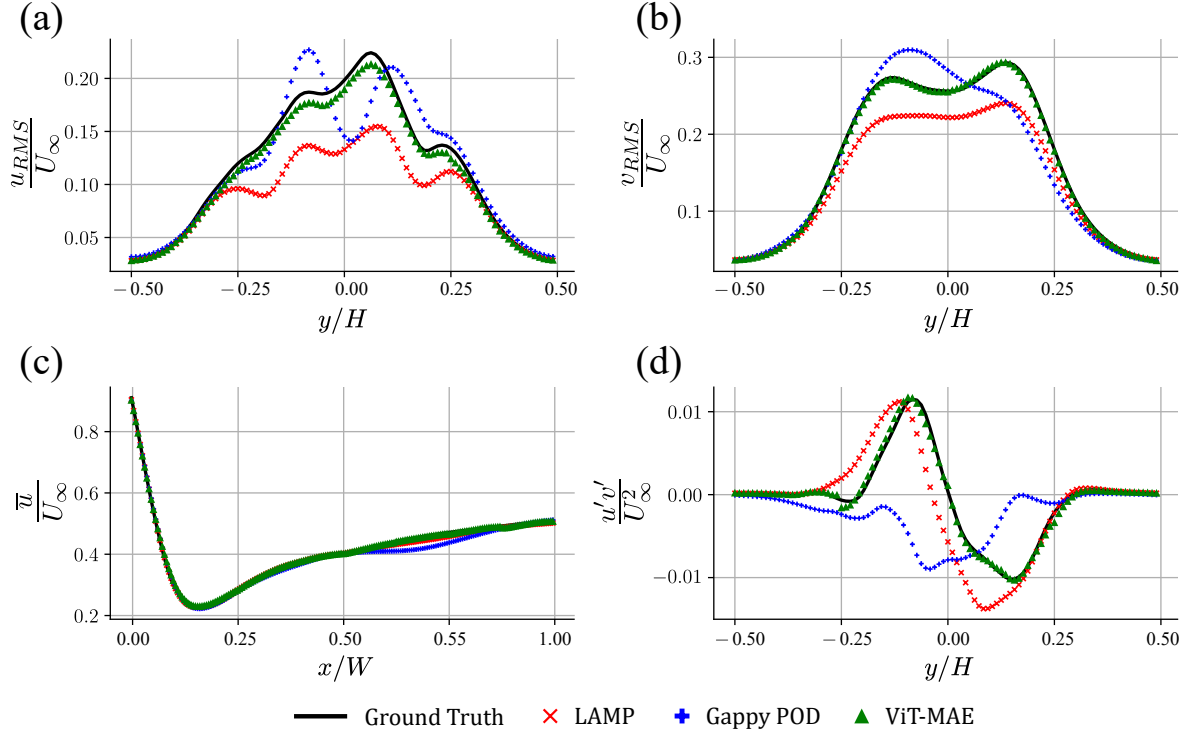


Figure 4.15: Distributions calculated in the same way as in Fig. 4.3 – also along the lines $x/W = 0.625$ and $y = 0$ – from the velocity fields reconstructed by the lowest- $\mathcal{L}^{\text{pred}}$ model variants of each class: LAMP (red) with $P = 48, D = 300$, Gappy POD (blue) with $D = 10$ and ViT-MAE (green) with the *reference architecture* (Table 3.1) and $P = 24$.

inconsistent performance overall (although each fails worst on different tasks), whereas ViT performs consistently across all 6 tests.

4.2.9 Speed Comparison

The training and inference times are reported in Table 4.3. Gappy POD and LAMP are unsurprisingly two to three orders of magnitude faster to train, without even utilising accelerated hardware. The inference frequency of ViT-MAE $\approx 300\text{Hz}$ makes it feasible for use in low-speed separation control [1], but not for high-speed jet control [45] or cavity flow control [46].

Table 4.3: Training and inference time for the three models used in the results section. Training time is reported for the dataset with 8,000 samples for comparison between the three methods (and scales linearly with the number of snapshots) and inference time is calculated for a single sample without batching. An AMD Ryzen Threadripper PRO 5995WX (64-core, 128-thread) CPU is used for the Gappy POD and LAMP results and ViT-MAE uses a single NVIDIA GeForce RTX 4090 GPU (24 GB VRAM).

P	Training Time	Inference Time (s)
Gappy POD	1 s	8×10^{-4}
LAMP	8 s	9×10^{-5}
ViT-MAE	63 min	3×10^{-3}

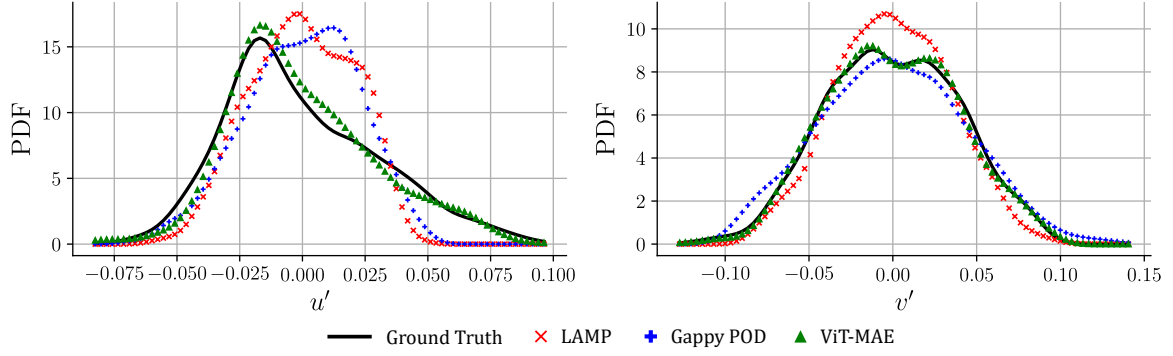


Figure 4.16: Probability density functions for (a) u' and (b) v' , calculated as in Fig. 4.4 at the point $x/W = 0.65, y = 0$. (a) shows a very positive skew, indicating the presence of high bursts of increased u . For (b), due to the symmetry of the setup about the line $y = 0$, the v' distribution should tend to a perfectly symmetrical distribution with the addition of more validation samples.

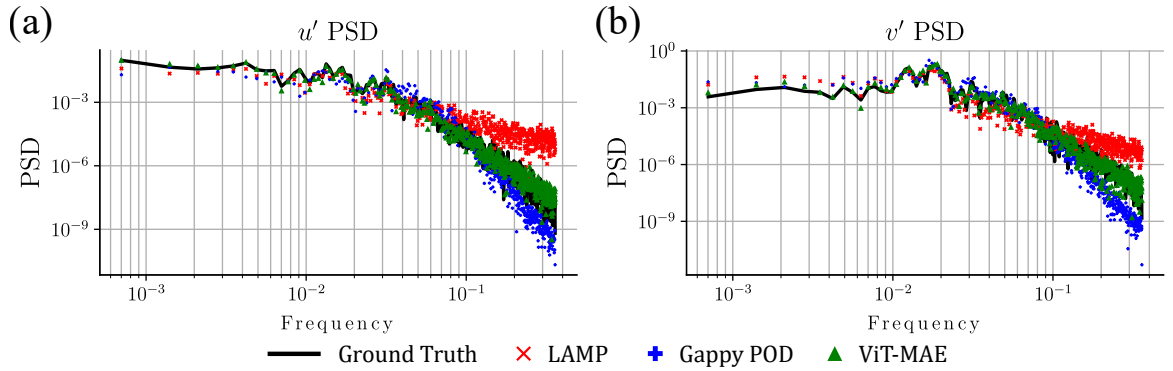


Figure 4.17: Power spectral density of u' and v' for the reconstructed fields, calculated as in Fig. 4.5 at the point $x/W = 0.65, y = 0$.

4.3 Dataset 3 (ViT)

Dataset 3 poses an even greater challenge. The flow has a much higher Reynolds number ($Re = 6,000$ compared to $Re = 200$ for Dataset 2) and visual inspection of the field plots of the three datasets (Fig. 3.2) shows a significant increase in the complexity of the flow structures. Compounding this, there are also far fewer snapshots available (5,000 total instead of 54,000). This ultimately puts a cap on the performance of the ViT-MAEs trained, as will be shown.

4.3.1 POD Reconstruction Loss

Although POD is not used on Dataset 3, $\mathcal{L}^{\text{POD}}$ is shown (Fig. 4.18) to demonstrate reduced correlations strength between regions compared to the previous two datasets. Firstly, Fig. 4.18 (a) shows a significant increase in the number of modes required to reconstruct the field accurately: the global POD basis reconstructs very poorly: $\mathcal{L}^{\text{POD}} = 0.74$ with 100 modes. Using smaller patches improves the reconstruction gradually, except for $P = 8$ which has an inflection in the $\mathcal{L}^{\text{POD}}$ vs d curve, leading it to decrease significantly, however this is because the compression factor is almost unity, defeating the purpose of the autoencoder – if the number of modes reaches the patch dimension ($d_P/d = 1$), the autoencoder can reconstruct with zero error but performs

no compression¹.

This can also be understood in terms of compression (Fig. 4.18 (b)). With the previous datasets, the fact that larger patches can obtain the same accuracy with a higher compression factor shows that there are meaningful correlations learned between neighbouring patches – this translates to larger patches more efficiently reconstructing the on the validation data. With Dataset 3, however, the linear correlations learned on the training data do not translate to the test data, causing the collapse of the curves for each P onto approximately one continuous curve. There is even a slight overlap between curves of consecutive patch sizes, indicating that, to the left of each intersection, the smaller patch size *more efficiently* captures the validation flow state. The fact that neighbouring patches are now no longer linearly correlated is a fundamental shift and ViT-MAEs have to rely on nonlinear relationships between patches (and groups of patches) to make predictions beyond just the mean flow.

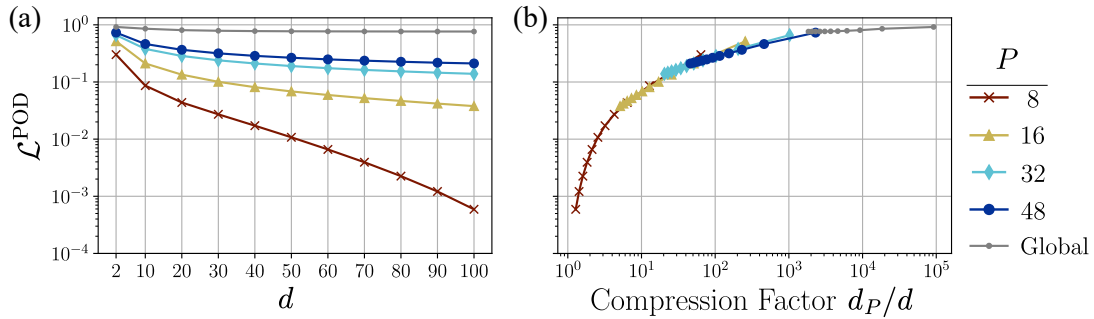


Figure 4.18: Patch-wise POD reconstruction loss plotted against (a) number of POD modes (b) compression factor. In contrast to the equivalent plots for datasets 1 and 2, (a) shows a massive increase in the number of modes required to reconstruct with the same accuracy. In (b), the curves from successive patch sizes cross over, showing that, to ensure low enough $\mathcal{L}^{\text{POD}}$, it becomes more efficient to use smaller patches. The flow in distant regions have such low correlation that it is more efficient to split up the domain and represent them with independent bases

4.3.2 Mean-Flow Plateau

Due to the complexity of the flow, training runs on this domain, across a *range* of architectures, all produced loss curves which plateaued within the range $\mathcal{L}^{\text{pred}} \in [0.55, 0.75]$ (see Fig. 4.19 (a)). The fact that the *training* loss curves go below this level shows that it was not due to a lack in model capacity. Looking at an example reconstruction (Fig. 4.19 (b)), the reconstructed flows resemble the mean flow, and calculating what the expected loss is if the model were to just predict the mean flow of the training data gives a value of 0.55, which sits at the lower limit of where the models plateau, further supporting the hypothesis. Crucially, for all of these tests, the domain used was $\Omega^{\text{val}} = \Omega^{\text{train}} = [10, 37] \times [-3.9, 3.9]$.

¹One can consider the extreme case occurs in with $P = 1$ (and assuming only one channel, $C = 1$). Only one mode is needed to achieve perfect reconstruction, where each POD basis is just a scalar representing the pixel brightness, but whilst perfect reconstruction is achieved, nothing is learned about the field. The attention mechanism, which scales with the square of the number of patches, N^2 , would also become prohibitively expensive.

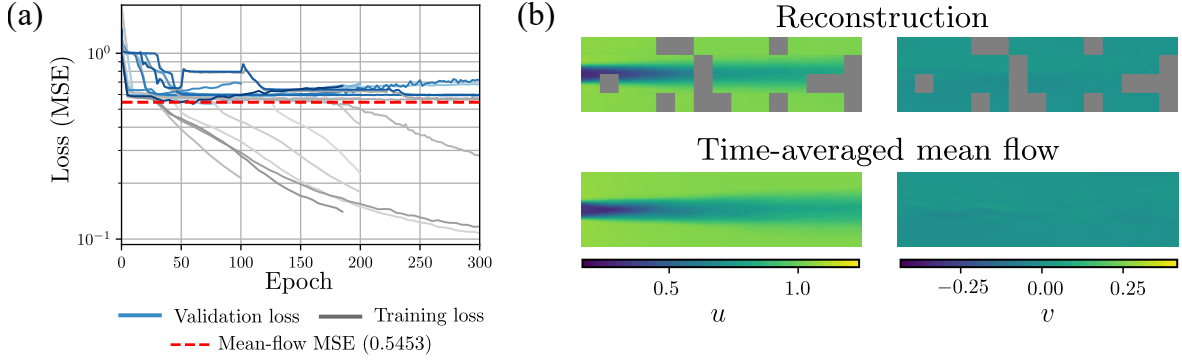


Figure 4.19: Loss plateau experienced by models of varying architectures. (a) shows the training (grey) and validation (blue) loss curves for a range of models – none of them improve upon the loss if one were to just predict the mean flow (shown by the red dashed line). Hyperparameters d, N_l, N_h, P , number of epochs, learning rate and weight decay were all varied. (b) shows an example snapshot reconstructed by one of these models, which can be compared with the actual mean flow calculated on the training data – they appear very similar.

4.3.3 Domain Size

This domain was then shrunk to a third the width, starting from the same x -position (i.e. $\Omega^{\text{val}} = \Omega^{\text{train}} = [10, 19] \times [-3.9, 3.9]$), and this was finally able to drop below the mean-flow MSE – it achieved $\mathcal{L}^{\text{pred}} = 0.29$, below the mean-flow MSE of the new domain (0.57). To verify that this is not just because this first third was an easier region to predict, models were trained on the other two remaining thirds (namely shifting $\Omega^{\text{val}}, \Omega^{\text{train}}$ 9 and 18 units rightwards), and as before, $\mathcal{L}^{\text{pred}} < \text{MSE}_{\text{mean-flow}}$, ($0.57 < 0.60$) and ($0.42 < 0.53$). The models perform better if the domain is first divided and multiple models are trained on each section, regardless of position.

Whilst the exact cause is not known for certain, it is hypothesised that it is in part related to the increased difficulty of optimising the attention mechanism on a larger domain, given the limited number of samples: the size of the attention matrix scales with N^2 (in this case, a factor of 9). The $\mathbf{W}^Q, \mathbf{W}^K$ and $\mathbf{W}^V$ matrices all receive gradients accumulated over $O(N^2)$ patch pair interactions. In a chaotic field, which has such weak long-range correlations, most of these interactions will carry no meaningful signal, acting as noise to dilute the meaningful predictive signal, and this makes it difficult to find the generalisation solution. Over-fitting, by contrast, is a far easier optimisation target, where the model just needs to associate input and output patterns on the training data. Fig. 4.20 (a) shows the loss decrease with an increasing number of training samples. $\mathcal{L}^{\text{pred}}$ decreases significantly with the number of samples and has not plateaued by the maximum number, indicating that the poor performance is largely due to the size of the dataset.

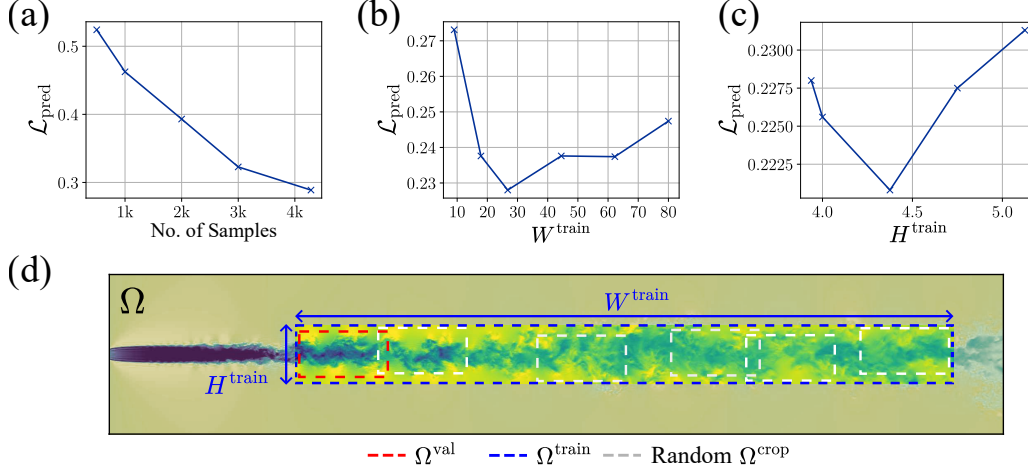


Figure 4.20: The effect of the training domain Ω_{train} on the loss. (a) shows how increasing the number of samples monotonically decreases the loss, (b) and (c) show the decrease in loss that can be obtained by increasing the width and height of the training domain respectively, and (d) illustrates the random crop technique. The validation domain Ω^{val} is fixed and then training samples Ω^{crop} are uniformly sampled from Ω^{train} .

4.3.4 Hyperparameter Tuning

With loss below the plateau, the optimal patch size was determined to be $P = 18$. Further hyperparameter tuning across the same hyperparameters as in Dataset 2 yielded no more than a 3% decrease in $\mathcal{L}^{\text{pred}}$, beyond that of the base model given in Appendix B.

Table 4.4: $\mathcal{L}^{\text{pred}}$ is calculated for ViT-MAEs with a range of patch sizes, demonstrating that $P = 18$ is optimal. Results quoted are for *random* patch arrangements.

P	12	18	24	36
$\mathcal{L}^{\text{pred}}$	0.31	0.29	0.31	0.35

4.3.5 Data Augmentation

Regarding data augmentation, vertical flipping is already applied to the previous experiments to artificially double the number of samples. The other avenue explored was using flow further along the wake as pre-training data before fine-tuning on the validation domain, namely employing the random cropping procedure explained in Section 3.2.4: A fixed $\Omega^{\text{val}} = [10, 19] \times [-3.9, 3.9]$ is used, and $\Omega^{\text{train}} = [10, W^{\text{train}}] \times [-3.9, 3.9]$, where the width W^{train} is varied. The results of this are shown in Fig. 4.20 (b) which demonstrates that increasing the length of the training domain lowers $\mathcal{L}^{\text{pred}}$ and that the optimal W^{train} is 26.75m, beyond which $\mathcal{L}^{\text{pred}}$ increases again – it appears that far downstream, the statistics of the flow vary too significantly to further increase performance. The same process is then repeated for the y -direction, to find the optimal $H^{\text{train}} = 8.75$, however the benefit is smaller – there is too large of a difference in the flow distributions if the domain is shifted significantly in y . In total, random cropping reduces loss by 19%. The quality of the reconstructions does not change noticeably because of the limited data, but from a better starting point, random cropping could provide meaningful benefit.

4.3.6 Masked Reconstruction

As seen already on Dataset 2, reconstruction loss increases with distance from the input windows. To demonstrate this on Dataset 3, Fig. 4.21 shows the reconstructions from input windows of three different widths. The reconstructed fields capture less of the detail in the wake outline as the width increases and this is reflected in plots below, which show the average reconstruction MSE calculated over all snapshots, the height and channel dimension – the maximum MSE in $c > b > a$. Each curve shows a sharp increase moving away from the input window and then a decrease as the x -position approaches the next window.

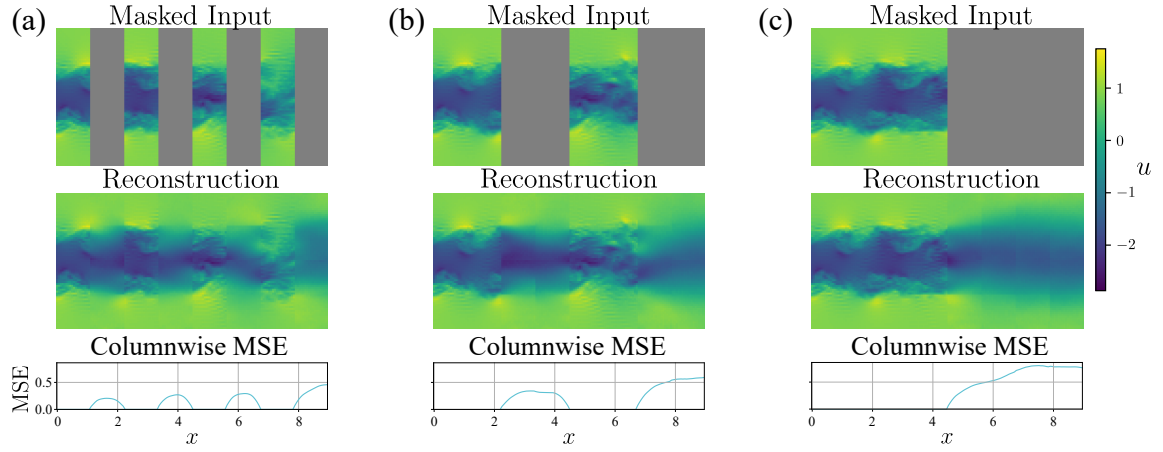


Figure 4.21: ViT-MAE reconstruction on Dataset 3 with input windows of equal widths and spacing of length (a) 1.125 m (b) 2.25 m (c) 4.5 m. The input is shown on the top row, with the reconstruction below and columnwise MSE below that.

5 - Conclusion

5.1 Summary of Thesis Achievements

Revisiting the four original aims, stated in Section 1.3, it is clear to see that these have all been achieved to various degrees. Regarding low-loss reconstructions, LAMP is shown to almost flawlessly reconstruct clean snapshots of Dataset 1, with certain flow distributions matching to within numerical precision. Then on Dataset 2, where LAMP struggles to capture key details of the chaotic flow, ViT-MAE demonstrates that it too can reconstruct distributions in masked regions of the test data (up to a certain distance downstream of the input window). Whilst the raw model introduces artefacts from the patching process, these can be mostly eliminated, with an additional U-Net block. In terms of data requirements, LAMP achieves the performance stated above with only 100 input samples and applying vertical flipping to Datasets 2 and 3 doubles the effective number of training samples. Ultimately, Dataset 3 has far too few snapshots to produce faithful reconstructions at any distance from the input sensors, however random cropping is identified as reducing the error by 19%. Robustness to noise is also thoroughly explored for both LAMP and ViT-MAE, with good performance on 30 dB noise as a key target – a proxy for experimental uncertainty. LAMP was able to reconstruct Dataset 1 snapshots with less error than even that of the noisy input data itself (simultaneously reconstructing and filtering), and this is due to the POD basis used; with ViT-MAE on Dataset 2, the clean reconstruction error was already higher than the noise added, but training on noisy data was able to bring down the additional error close to that of the noise-free reconstruction. Calculating the inference speed on Dataset 2, ViT-MAE was able to reconstruct at frequency of 300 Hz, suggesting that it would be a viable option for active flow control problems such as in [1].

More broadly, it is shown that the ViT-MAE architecture can be used to reconstruct static flow-fields from significantly masked inputs; the attention mechanism learns complex nonlinear relationships spanning the whole domain. To gradually build up to the full ViT-MAE model, the new LAMP model is developed; it can generate attention maps, developing intuition about the attention mechanism unique to Transformer-based models, and provides a more relevant benchmark to patch-based models than Gappy POD. Considerations important to fluid dynamics (e.g. frequency analysis and continuity) are also explored to assess the relevance of ViT-MAE in scientific applications: velocity distributions show high agreement with the ground truth and patch-boundary discontinuities can be removed with a post-processing U-Net block. At the same time, the limitations of the model are explored, predominantly the existence of sparse but severe error spikes, with solutions proposed in Section 5.3.

5.2 Applications

Based on the results, ViT-MAE is a viable candidate for real-time flow reconstruction from PIV windows, subject to resolution of the intermittent error spikes. Three aspects of the experimental setting were specifically considered: the use of rectangular input windows mirrors a realistic PIV setup; the effects of experimental noise levels are shown to be manageable; and inference speed. The primary barrier to implementing this is the availability of data – results for Dataset 3 demonstrate that this is a significant constraint, so acquiring additional data is the

most immediate requirement. Otherwise, the methods and analysis presented here make this a tractable next step.

5.3 Future Work

Analysis in Section 4.2.5 suggests that large error spikes are, in large part, due to the sensitivity of the flow to small perturbations. One potential solution to provide the model with more data, whilst still operating within the constraints of the PIV window input, is to augment the input state with time history of previous snapshots. In the case of Dataset 2, this would allow the model to better differentiate between similar flow states, reducing the information bottleneck.

Physically, convection-dominated flows (approximated as hyperbolic partial differential equations) are characterised by a finite speed of information flow. Therefore, the flow state downstream should be better predicted by the state of the flow upstream at *previous* snapshots in time than at the current time. For Dataset 3, a visual inspection of the flow evolution from a *Lagrangian* frame of reference shows a far slower-evolving field. The model currently learns correlations within a quasi-periodic signal, whereas if it is also provided time history, ViT-MAE is expected to utilise flow *dynamics* to better predict downstream flow, learning the underlying principles governing fluid flow. Turbulence is an intrinsically temporal phenomenon, so neglecting temporal information severely handicaps the model. The first priority of future work will thus be to investigate the addition of previous snapshots. Transformers were originally designed for sequential data [24], so there is already precedent.

Future work could also investigate the use of attention maps to guide sensor placement. In LAMP, attention maps give an average score of how well each patch can predict the rest of the patches and the full log-attention matrix, $\mathbf{A}$, gives an even richer picture, with the score for each pair-wise prediction. Naively placing sensors in order of attention score risks redundancy, since neighbouring patches are expected to be highly correlated; a greedy selection algorithm [47] would account for this diminishing return. Furthermore, attention maps are specific to the latent dimensionality, d , used, so utilising attention maps generated across a *range* of d is predicted to enable reconstruction accuracy of features across a range of spatial scales, achieving fine-scale resolution whilst ensuring robustness to noise. If achieved, this analysis should be expanded to include the $\mathbf{A}$ matrices present within the ViT-MAE, to scale this idea to more complicated flows than what LAMP can reconstruct.

Finally, comprehensive optimisation of model architecture is left to future work and should be tailored to a specific use-case. Knowledge about a particular experimental setup (e.g. noise intensity and Reynolds number) and requirements from the wider application (such as inference speed and solution smoothness) will all result in different optimal solutions. As demonstrated with the U-Net experimentation, the ViT-MAE architecture is a powerful base from which to expand.

Bibliography

- [1] Chengwei Xia et al. “Active flow control for bluff body drag reduction using reinforcement learning with partial measurements”. In: *Journal of Fluid Mechanics* 981 (2024), A17.
- [2] Ferran Alet et al. *Skillful joint probabilistic weather forecasting from marginals*. 2025. arXiv: 2506.10772 [cs.LG].
- [3] Markus Raffel et al. *Particle Image Velocimetry: A Practical Guide*. 3rd. Springer, 2018.
- [4] Albert Tarantola. *Inverse Problem Theory and Methods for Model Parameter Estimation*. SIAM, 2005.
- [5] Donald L. Loving and S. Katzoff. *The Fluorescent-Oil Film Method and Other Techniques for Boundary-Layer Flow Visualization*. Tech. rep. NASA-MEMO-3-17-59L. NASA Langley Research Center, 1959.
- [6] Francis De Voogt and Bharathram Ganapathisubramani. “Determination of unsteady wing loading using tuft visualization”. In: *Experiments in Fluids* 65.10 (2024), p. 148.
- [7] Martin W. Hess, Marco Tezzele, and Gianluigi Rozza. “Overview and Motivation”. In: *Advanced Reduced Order Methods and Applications in Computational Fluid Dynamics*. SIAM, 2022, pp. 1–12.
- [8] Elise Özalp, Andrea Nóvoa, and Luca Magri. “Real-time forecasting of chaotic dynamics from sparse data and autoencoders”. In: *Computer Methods in Applied Mechanics and Engineering* 450 (2026), p. 118600.
- [9] Yann LeCun et al. “Backpropagation applied to handwritten zip code recognition”. In: *Neural Computation* 1.4 (1989), pp. 541–551.
- [10] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. “ImageNet classification with deep convolutional neural networks”. In: *Proc. of NeurIPS*. 2012.
- [11] Ian Goodfellow et al. “Generative adversarial nets”. In: *Proc. of NeurIPS*. 2014.
- [12] Jonathan Ho, Ajay Jain, and Pieter Abbeel. “Denoising diffusion probabilistic models”. In: *Proc. of NeurIPS*. 2020.
- [13] Yang Song et al. “Score-based generative modeling through stochastic differential equations”. In: *Proc. of ICLR*. 2021.
- [14] Alexey Dosovitskiy. “An Image Is Worth 16×16 Words: Transformers for Image Recognition at Scale”. In: *Proc. of IEEE/CVF CVPR*. 2021, pp. 45–67.
- [15] G. Berkooz et al. “The Proper Orthogonal Decomposition in the Analysis of Turbulent Flows”. In: *Annual Review of Fluid Mechanics* 25 (1993), pp. 539–575.
- [16] Richard Everson and Lawrence Sirovich. “Karhunen–Loève procedure for gappy data”. In: *Journal of the Optical Society of America A* 12.8 (1995), pp. 1657–1664.
- [17] A. Güemes, S. Discetti, and A. Ianiro. “Sensing the turbulent large-scale motions with their wall signature”. In: *Physics of Fluids* 31.12 (2019), p. 125112.
- [18] Nguyen Anh Khoa Doan, Alberto Racca, and Luca Magri. “Convolutional Autoencoder for the Spatiotemporal Latent Representation of Turbulence”. In: *Proc. of ICCS 2023*. Vol. 10476. Lecture Notes in Computer Science. Springer, 2023.

- [19] Kai Fukami, Koji Fukagata, and Kunihiko Taira. “Super-resolution reconstruction of turbulent flows with machine learning”. In: *Journal of Fluid Mechanics* 870 (2019), pp. 106–120.
- [20] Feng Xie, Pei Lu, and Xiaoyong Liu. “Multi-scale convolutional attention network for lightweight image super-resolution”. In: *Journal of Visual Communication and Image Representation* 95 (2023), p. 103889.
- [21] Andrea Beck, David Flad, and Claus-Dieter Munz. “Deep neural networks for data-driven LES closure models”. In: *Journal of Computational Physics* 398 (2019).
- [22] Mustafa Z. Yousif et al. “A deep-learning approach for reconstructing 3D turbulent flows from 2D observation data”. In: *Scientific Reports* 13 (2023), p. 2529.
- [23] Mustafa Z. Yousif et al. *Flow Reconstruction Using Spatially Restricted Domains Based on Enhanced Super-Resolution Generative Adversarial Networks*. 2024. arXiv: 2408.01658 [physics.flu-dyn].
- [24] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).
- [25] Kaiming He et al. “Masked autoencoders are scalable vision learners”. In: *Proc. of IEEE/CVF CVPR*. 2022, pp. 16000–16009.
- [26] William Peebles and Saining Xie. “Scalable Diffusion Models with Transformers”. In: *Proc. of IEEE/CVF ICCV*. 2023, pp. 4172–4182.
- [27] Aditya Ramesh et al. “Zero-shot text-to-image generation”. In: *Proc. of ICML*. 2021.
- [28] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning representations by back-propagating errors”. In: *Nature* 323 (1986), pp. 533–536.
- [29] George Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of Control, Signals and Systems* 2 (1989), pp. 303–314.
- [30] Xavier Glorot and Yoshua Bengio. “Understanding the difficulty of training deep feedforward neural networks”. In: *Proc. of AISTATS*. 2010.
- [31] Karen Willcox. “Unsteady flow sensing and estimation via the gappy proper orthogonal decomposition”. In: *Computers & Fluids* 35.2 (2006), pp. 208–226.
- [32] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *Proc. of IEEE/CVF CVPR*. 2016, pp. 770–778.
- [33] Tri Dao et al. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”. In: *Proc. of NeurIPS*. 2022.
- [34] Ben Eze, Luca Magri, and Andrea Nóvoa. *Latent Attention on Masked Patches for Flow Reconstruction*. 2026. arXiv: 2603.02028 [cs.LG].
- [35] Yaxin Mo, Tullio Traverso, and Luca Magri. “Decoder decomposition for the analysis of the latent space of nonlinear autoencoders with wind-tunnel experimental data”. In: *Data-Centric Engineering* 5 (2024), e38.
- [36] Elif Bekoglu, Nikos Bempedelis, and Konstantinos Steiros. “Formation of turbulent secondary vortex street in absence of vortex shedding instability”. In: *Journal of Fluid Mechanics* 1024 (2025), A7.

- [37] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization”. In: *Proc. of ICLR*. 2019.
- [38] Ilya Loshchilov and Frank Hutter. “SGDR: Stochastic Gradient Descent with Warm Restarts”. In: *Proc. of ICLR*. 2017.
- [39] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. “U-Net: Convolutional Networks for Biomedical Image Segmentation”. In: *Proc. of MICCAI*. Vol. 9351. Lecture Notes in Computer Science. Springer, 2015, pp. 234–241.
- [40] Kai Fukami, Koji Fukagata, and Kunihiro Taira. “Machine-learning-based spatio-temporal super resolution reconstruction of turbulent flows”. In: *Journal of Fluid Mechanics* 909 (2021), A9.
- [41] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations”. In: *Journal of Computational Physics* 378 (2019), pp. 686–707.
- [42] Steven J. Beresh. “Evaluation of PIV Uncertainties using Multiple Configurations and Processing Techniques”. In: *Proc. of 46th AIAA Aerospace Sciences Meeting and Exhibit*. AIAA Paper 2008-0239. Reno, NV, 2008.
- [43] Momchilo M. Zdravkovich. *Flow Around Circular Cylinders: Volume 1: Fundamentals*. Oxford University Press, 1997.
- [44] Prasanta Chandra Mahalanobis. “On the generalised distance in statistics”. In: *Proceedings of the National Institute of Sciences of India* 2.1 (1936), pp. 49–55.
- [45] Louis N. Cattafesta and Mark Sheplak. “Actuators for Active Flow Control”. In: *Annual Review of Fluid Mechanics* 43 (2011), pp. 247–272.
- [46] Michael A. Kegerise, Randolph H. Cabell, and Louis N. Cattafesta. “Real-Time Adaptive Control of Flow-Induced Cavity Tones”. In: *Proc. of 42nd AIAA Aerospace Sciences Meeting and Exhibit*. AIAA Paper 2004-0572. Reno, NV, 2004.
- [47] Krithika Manohar et al. “Data-Driven Sparse Sensor Placement for Reconstruction: Demonstrating the Benefits of Exploiting Known Patterns”. In: *IEEE Control Systems Magazine* 38.3 (2018), pp. 63–86.

I acknowledge the use of Claude (<https://claude.ai>) for help with initial research, debugging code and proofreading. The writing in this report is my own work.

A - Experimentation Framework

The following is an example config file for the *reference* architecture:

```
1 # yamll-language-server: $schema=./config.schema.json
2 description: Sweep over patch size and latent dimension for comparison with
   LAMP
3 Fig. 5 (16, 20).
4 meta:
5   from_load: false
6   experiments_dir: experiments
7   experiment_subdir: ParameterSweeps/P_Ne/sweep_0000
8   run_id: '0003'
9   run_dir: experiments/ParameterSweeps/P_Ne/sweep_0000/run_0003
10  git_hash: 0ed8dbd5c92b40a68b44f53ede54cf70da4d071e
11 data:
12   type: masked
13   sources:
14     - path: ../.cache/transfer/2cyl_s=1.5d,ofs=0.5,cfl=0.25/X.npy
15       train_slice: [[2000, null], [null, null, 2], [116, -76, 2], [null, null]]
16     - path: ../.cache/transfer/2cyl_s=1.5d,ofs=0,cfl=0.2/X.npy
17       train_slice: [[2000, null], [null, null, 2], [116, -76, 2], [null, null]]
18     - path: ../.cache/transfer/2cyl_s=1.5d,ofs=0,cfl=0.2,noisy/X.npy
19       train_slice: [[2000, null], [null, null, 2], [116, -76, 2], [null, null]]
20     - path: ../.cache/transfer/2cyl_s=1.5d,ofs=1,cfl=0.25/X.npy
21       train_slice:
22         train_slice: [[2000, null], [null, null, 2], [116, -76, 2], [null, null]]
23     - path: ../.cache/transfer/2cyl_s=1.5d,ofs=1.5,cfl=0.2/X.npy
24       train_slice:
25         train_slice: [[2000, null], [null, null, 2], [116, -76, 2], [null, null]]
26     - path: ../.cache/transfer/2cyl_s=1.5d,ofs=0,cfl=0.25,rk3/X.npy
27       train_slice: [[2000, null], [null, null, 2], [116, -76, 2], [null, null]]
28     - path: ../.cache/transfer/2cyl_s=1.5d,ofs=0,cfl=0.25/X.npy
29       train_slice: [[2000, 5000], [null, null, 2], [116, -76, 2], [null, null]]
30       val_slice: [[8000, null], [null, null, 2], [116, -76, 2], [null, null]]
31   patch_size: 24
32   frac_unmasked: 0.25
33   ceil_unmasked: true
34   num_workers: 4
35   augmentation:
36     horizontal_flip: false
37     vertical_flip: true
38     noise_snr_db: 120
39 model:
40   family: vit
41   subclass: autoencoder
42   params:
43     C: 2
44     H: 96
45     W: 192
46   architecture:
47     patch_embed:
48       P: 24
49       D: 384
```

```

50     blocks_enc:
51         Nl: 12
52         Nh: 6
53         Dh: 64
54         Dmlp: 1536
55         dropout: 0.0
56     enc_dec_bridge:
57         identity: false
58     blocks_dec:
59         Nl: 4
60         Nh: 4
61         Dh: 64
62         Dmlp: 1024
63         dropout: 0.0
64     patch_unembed:
65         P: 24
66         D: 256
67 training:
68     optimiser:
69         type: adamw
70         kwargs:
71             lr: 0.0005
72             weight_decay: 0.2
73     loss_function:
74         type: mse
75         kwargs: {}
76     scheduler:
77         type: cosine_annealing_lr
78         kwargs:
79             T_max: 700
80             eta_min: 1e-06
81     N_epochs: 700
82     batch_size: 128
83     save_interval: 2

```

Listing A.1: Run configuration for reference architecture

Fig. A.1 shows an example dashboard created for a parameter sweep, ensuring visibility of the training runs currently operating

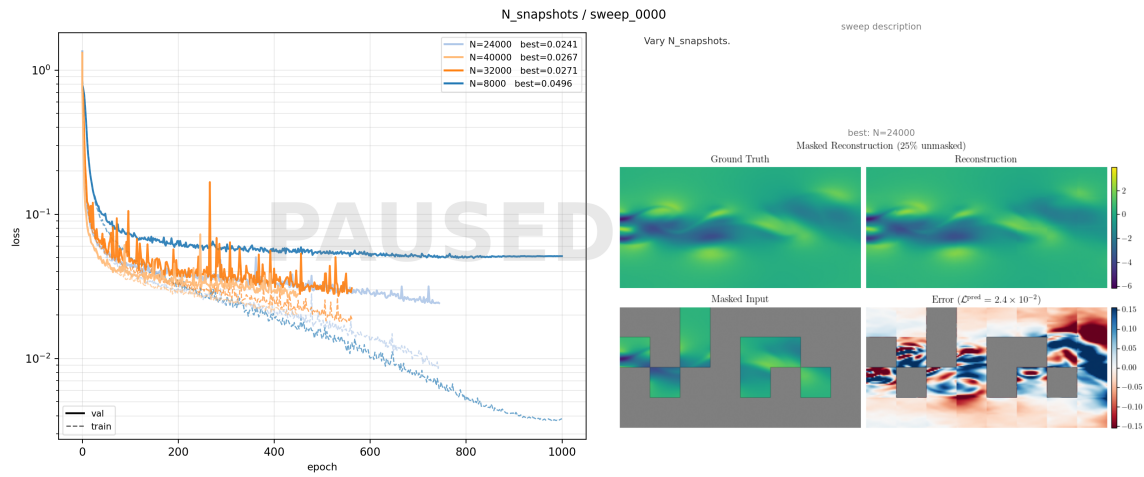


Figure A.1: Example dashboard showing the loss curves of all the models in a parameter sweep. The reconstruction from the lowest loss model is also shown. The script `run_dashboard.py` continuously runs and updates the figure `dashboard.png` and the PAUSED indicates that the script has reached the time limit.

B - Dataset 3 ViT-MAE Hyperparameters

Table B.1: Architecture hyperparameters used for the ViT training on Dataset 3. The total parameter count is 44,393,864

config		value
general	patch size	18
	dropout	0.0
encoder	no. of layers	8
	no. of heads	12
	head dim	64
	MLP dim	768
decoder	no. of layers	8
	no. of heads	8
	head dim	64
	MLP dim	768

Table B.2: Training hyperparameters used for the ViT training on Dataset 3.

config	value
optimiser	AdamW
scheduler	cosine annealing
learning rate (start)	1×10^{-4}
learning rate (end)	2×10^{-5}
weight decay	1×10^{-4}
epochs	1000
batch size	128
fraction unmasked	0.5

C - Distribution Correlation (R^2) Values

Table C.1: R^2 scores for flow-statistic reconstruction of LAMP. The R^2 value for the PSD is given for a linear and log scale.

Metric	LAMP
u_{rms} profile	1.000
v_{rms} profile	1.000
$\bar{u}$ centreline	1.000
$\overline{u'v'}$ profile	1.000
u' PDF	1.000
v' PDF	1.000
u' PSD	1.000
v' PSD	1.000
u' PSD (log)	0.986
v' PSD (log)	0.994

Table C.2: R^2 scores for flow-statistic reconstructions of Gappy POD, LAMP and ViT-MAE. The R^2 value for the PSD is given for a linear and log scale.

Metric	Gappy POD	LAMP	ViT-MAE
u_{rms} profile	0.865	0.646	0.997
v_{rms} profile	0.917	0.899	0.999
$\bar{u}$ centreline	0.978	0.997	0.999
$\overline{u'v'}$ profile	-0.672	0.625	0.992
u' PDF	0.580	0.550	0.987
v' PDF	0.977	0.940	0.995
u' PSD	0.520	0.689	0.973
v' PSD	0.507	0.779	0.978
u' PSD (log)	0.625	-0.000	0.925
v' PSD (log)	0.413	0.486	0.915